STS Brown Bag Series
Enterprise Claude Code: AI-Powered Development for the Federal Mission
Friday, April 3, 2026 | 12:00 - 1:00 PM ET

Presenters:
- Rudy Zauel, Sr. Software Developer
- Eric Chasin, Sr. Principal Architect

Audience: ~24 STS software engineers

[simple]

ACT 1

Agentic Concepts

What is this stuff and why should I care?

© Simple Technology Solutions, Inc.

What Is a Model?

[simple]

It's not Google (search engine), not a hard drive (database), not a calculator — it's a **very well-read colleague** who learned by reading millions of codebases and books

Ask it the same question twice, you might get two different answers — like asking two senior engineers the same design question

It **reasons through problems**, not looks up answers — predicting good code requires understanding what good code looks like

It can write your app but can't add large numbers reliably — brilliant with patterns, unreliable with precision

It graduated on a specific date and hasn't read anything since — anything after its training cutoff is a blind spot

© Simple Technology Solutions, Inc.

It's Not What You Think It Is

Most people's first instinct is to treat an LLM like a search engine or a database — you type a question, it looks up the answer. That's not what's happening. The model is a transformer neural network that has absorbed statistical patterns from enormous amounts of code and text during training. When you give it a prompt, it generates what is most likely to come next, one token at a time.

How It Actually Works

- Transformer architecture: The model processes input through layers of attention mechanisms that weigh relationships between every token in the context. This is what lets it understand code structure, follow long chains of logic, and maintain coherence across thousands of tokens.
- Next-token prediction: At its core, the model predicts the probability distribution over all possible next tokens, then samples from that distribution. This is why the same prompt can produce different outputs — it's sampling from probabilities, not looking up a fixed answer.
- Emergent reasoning: Ilya Sutskever's insight — predicting the ending of a detective novel requires understanding the entire plot. When the "next token" requires genuine insight to predict correctly, the model has to have internalized that reasoning capability. Predicting good code requires understanding what good code looks like.

What It's Good At vs. What It Struggles With

Strengths:

- Pattern recognition and code generation — It has seen millions of codebases. It recognizes architectural patterns, API conventions, framework idioms, and common bug patterns. It can generate idiomatic code in dozens of languages.
- Reasoning through well-represented problems — If the type of problem appeared frequently in training data (web apps, REST APIs, data transformations, algorithm implementations), the model reasons through it effectively.
- Summarization and synthesis — Condensing large amounts of information, explaining code, comparing approaches, generating documentation.
- Brainstorming and exploration — Generating multiple approaches to a problem, identifying edge cases, suggesting alternative architectures.

Weaknesses:

- Precise arithmetic and counting — The model approximates; it doesn't calculate. Don't ask it to count tokens, add large numbers, or do exact math without a tool.
- Guaranteed factual accuracy — There is no internal fact-check step. The model optimizes for fluency (what sounds right), not accuracy (what is right). When fluency and accuracy conflict, fluency wins. This is why hallucination happens.
- Knowledge after training cutoff — The model's parametric knowledge is frozen at its cutoff date. Anything after that date is unknown unless you supply it in context. Opus 4.6 cutoff: May 2025. Sonnet 4.6: August 2025.
- Deterministic behavior — Same prompt, different run, potentially different output. This is a feature of the probabilistic architecture, not a bug — but it means you need verification, not trust.

The Three-Column Mental Model

Database	Search Engine	LLM
Key in, exact value out	Query in, existing documents out	Prompt in, generated response out
Deterministic	Retrieval-based	Probabilistic reasoning
Looks up stored data	Finds existing content	Creates new content from patterns

This comparison is the key reframe for your audience. Engineers are used to deterministic systems. An LLM is fundamentally probabilistic — and that changes everything about how you verify its output.

Why This Matters for Agentic Engineering

The probabilistic nature of models is exactly why agentic engineering exists. If models were deterministic and always correct, you'd just call them once and trust the output. Because they're probabilistic:

- You need verification gates (TypeScript compilation, build checks, test suites) to catch the output that sounds right but isn't

- You need the agent loop (think, act, observe, repeat) so the model can self-correct when verification fails
- You need fresh context to avoid degradation — the model's attention mechanism works best when context is focused, not bloated
- You need CLAUDE.md and system prompts to constrain the model's behavior into consistent, predictable patterns

The entire agentic engineering discipline is built on the answer to this slide's question: it's a powerful reasoning engine that needs structure, verification, and good context to produce reliable output.

How Does a Model Know Things?

[simple]

Two kinds of knowledge: **what it learned in school** (training data — broad but frozen on graduation day) and **the briefing you hand it before the meeting** (context — temporary, gone when the session ends)

It stores knowledge as **math, not facts** — "Michael Jordan is the GOAT" isn't in a database, it's that "Michael Jordan" and "greatest basketball player" are close together in vector space

It **graduated on a specific date** — Opus: May 2025, Sonnet: August 2025. Ask about anything after that and it's guessing or making things up

It **confidently makes things up** — there's no internal fact-checker. If it sounds right, the model says it, even if it's wrong

You fill the gaps — CLAUDE.md, file reads, and tool outputs are how you give it knowledge it doesn't have

© Simple Technology Solutions, Inc.

Two Ways a Model "Knows" Something

There are exactly two mechanisms, and understanding the distinction is critical for engineering with AI agents.

1. Parametric Knowledge (Training Data)

- During training, the model processes enormous amounts of code and text. The patterns it learns get encoded into its neural network weights — billions of numerical parameters.
- These weights represent knowledge as high-dimensional vectors — lists of hundreds or thousands of numbers that encode meaning as positions in mathematical space. Every word, concept, and pattern the model has learned lives somewhere in this vector space.
- Vector arithmetic captures relationships. The classic example: the vector for "king" minus "man" plus "woman" lands near the vector for "queen." Not exactly on it — these are approximate, probabilistic representations — but close enough that the model has learned the relationship between gendered and royal concepts purely from patterns in text. Similarly, vector("Paris") - vector("France") + vector("Germany") ≈ vector("Berlin").
- This is how the model "knows" things nobody explicitly wrote down. There is no database row that says "Michael Jordan is the greatest basketball player of all time." Instead, the vector for "Michael Jordan" sits in a region of vector space near "basketball," "championships," "GOAT," "greatness," and "NBA

legend." When you ask the model who the GOAT is, it's not looking up a stored answer — it's navigating vector space where "Michael Jordan" and "greatest basketball player" are geometrically close because millions of texts placed those concepts together. The answer emerges from the math of proximity, not from a fact table.
- This means the model doesn't store facts like a database stores rows. It stores relationships between concepts as geometric distances and directions in high-dimensional space. "Knowing" that Python is a programming language means Python's vector is near other programming languages and far from snakes — the model learned the context determines which meaning applies.
- This is the model's "education." It covers most programming languages, frameworks, APIs, design patterns, and general world knowledge up to the training cutoff.
- It's broad — the model has seen millions of codebases, documentation pages, and Stack Overflow answers.
- It's frozen — the weights don't change after training. The model cannot learn new facts, update its understanding of an API, or know about a library released last week.
- Think of it as a very well-read colleague who graduated on the cutoff date and hasn't read anything since.

2. In-Context Learning (Session Knowledge)

- Information you provide in the conversation: system prompts, CLAUDE.md, file contents read by tools, previous messages, tool results.
- The model adapts its behavior based on this information within the session — no weight updates, purely attention-based.
- This is the briefing document you hand someone before a meeting. It's temporary and disappears entirely when the session ends.
- In-context information can override parametric knowledge — if CLAUDE.md says "use Tailwind v4 CSS-based config, NOT tailwind.config.js," the model follows that even though its training data is full of tailwind.config.js examples.
- The limit is the context window size. Everything the model can "see" right now has to fit within this window.

Why Models Hallucinate

- The model generates text token by token, optimizing for "what is most likely to come next" based on the patterns it learned.
- There is no internal verification step. The model doesn't check whether its output is factually correct, whether a function actually exists, or whether an API endpoint returns the shape it claims.
- When fluency and accuracy conflict, fluency wins. The model will confidently generate a plausible-looking API call to an endpoint that doesn't exist, because the pattern of API calls is well-represented in training data even if that specific endpoint isn't.
- This is not a bug that will be fixed — it's a fundamental property of how probabilistic generation works. The engineering response is verification, not trust.

The Cutoff Problem

Model	Knowledge Cutoff	Implication
Opus 4.6	May 2025	Reliable for frameworks/APIs stable before mid-2025

| Sonnet 4.6 | August 2025 | Slightly more current, still months behind |
| Haiku 4.5 | — | Smallest model, fastest, same cutoff constraints |

- If a library released a breaking change after the cutoff, the model will generate code for the old API.
- If a new framework or tool launched after the cutoff, the model either doesn't know it exists or confuses it with something similar.
- This is why context matters so much: you supply the model with current documentation, your project's actual code, and explicit instructions about which versions and patterns to use.

How Context Fills the Gap

The practical engineering response to the knowledge gap:

- CLAUDE.md — Encodes your project's conventions, tech stack versions, and patterns. Overrides stale parametric knowledge with current project reality.
- File reads (Read, Glob, Grep tools) — The agent reads your actual codebase. It doesn't guess what your code looks like — it reads it. This is live, accurate information that supplements training data.
- Tool outputs (Bash results) — When the agent runs npm run build and gets an error, that error message is current, real-world information. The agent reasons about actual compiler output, not imagined behavior.
- Conversation history — Previous messages in the session provide continuity. If you corrected the model's approach earlier, that correction persists in context for the rest of the session.

The Two-Layer Diagram

Think of the model's knowledge as two layers:

```
┌──────────────────────────────────────────┐
│  Context Window (temporary, session-scoped)  │
│  CLAUDE.md + files + tool results + convo    │
│  ↓ Supplements & Overrides ↓                 │
├──────────────────────────────────────────┤
│  Parametric Knowledge (permanent, frozen)    │
│  Training data — broad but stops at cutoff   │
│                          ╳ cutoff  │
│                     "Here be dragons"│
└──────────────────────────────────────────┘
```

Everything above the line is what you control. Everything below the line is what the model came with. Your job as an agentic engineer is to make that top layer precise enough that the frozen bottom layer doesn't lead the model astray.

Why This Matters for Agentic Engineering

- Verification gates exist because of hallucination — you can't trust the model's self-assessment that its code is correct. You trust tsc, npm run build, and test suites.
- CLAUDE.md exists because of knowledge cutoffs — you encode current project reality so the model doesn't fall back on stale training data.

- Fresh context per session exists because in-context learning is temporary — each new session needs the full briefing. There's no accumulated learning between sessions.
- The entire agentic pipeline (research → spec → CLAUDE.md → build prompts → verification) is an engineering response to these two knowledge mechanisms and their limitations.

What Are the Differences Between the Claude Models?

[simple]

Haiku — The intern. Fast, cheap, good at simple tasks. You send it to scan files and fetch information

Sonnet — The senior engineer. Your daily workhorse — handles 90%+ of real coding work at a reasonable price

Opus — The principal architect. Slowest but smartest. Bring it in for hard problems, big-picture decisions, and agent teams

Like cloud compute tiers — **you don't run everything on the biggest server**. Match the model to the task

Claude Code does this automatically — quick lookups go to Haiku, real work goes to Sonnet or Opus

© Simple Technology Solutions, Inc.

The Model Hierarchy

Think of it like compute tiers in a cloud provider. You don't run everything on the largest instance — you match the model to the task. Claude's three models are designed to cover the full spectrum from quick lookups to deep architectural reasoning.

Haiku 4.5 — The Fast Thinker

- Cost: $1 input / $5 output per million tokens — the cheapest tier
- Context window: 200K tokens
- Max output: 64K tokens
- Speed: Fastest of the three
- Best for:
  - Subagent tasks (file discovery, classification, simple code reads)
  - Routing decisions (is this task simple or complex?)
  - Quick summarization and extraction
  - High-volume, low-complexity operations
- Not ideal for: Complex multi-step reasoning, large codebase refactoring, nuanced architectural decisions
- In Claude Code: Automatically used for subagents spawned by the main agent. When Claude Code needs to quickly scan a directory or classify a task, Haiku handles it without burning expensive tokens on a larger model.

Sonnet 4.6 — The Daily Driver

- Cost: $3 input / $15 output per million tokens
- Context window: 1M tokens (1,000,000)
- Max output: 64K tokens
- Speed: Fast — not as quick as Haiku, but responsive enough for interactive use
- Best for:
  - Day-to-day coding tasks (implementing features, fixing bugs, writing tests)
  - Code review and explanation
  - Multi-file edits with moderate complexity
  - The vast majority (90%+) of software engineering work
- Why it's the default: Best cost-to-quality ratio. Preferred over older models by 70% of developers in benchmarks. Capable enough for most tasks, cheap enough to use freely.
- In Claude Code: The default model for main implementation tasks. When you open Claude Code and start working, Sonnet is doing the heavy lifting unless you've explicitly selected Opus.

Opus 4.6 — The Deep Reasoner

- Cost: $5 input / $25 output per million tokens
- Context window: 1M tokens (1,000,000)
- Max output: 128K tokens — double the output ceiling of Haiku and Sonnet
- Speed: Moderate — noticeably slower than Sonnet, but the quality difference is measurable on hard problems
- Best for:
  - Complex architecture decisions spanning many files
  - Large codebase analysis and understanding
  - Agent teams (requires Opus — multiple coordinating agents)
  - Multi-step planning and reasoning chains
  - Tasks where getting it right the first time matters more than speed
- The 128K output ceiling: Opus can generate twice as much output in a single turn. This matters for large refactors, comprehensive test suites, or generating multiple files in sequence.
- In Claude Code: Selected explicitly by the user for complex work, or required for features like Agent Teams. Available with the 1M extended context option for massive codebase analysis.

Side-by-Side Comparison

	Haiku 4.5	Sonnet 4.6	Opus 4.6
Input cost (per MTok)	$1	$3	$5
Output cost (per MTok)	$5	$15	$25
Context window	200K	1M	1M
Max output	64K	64K	128K
Speed	Fastest	Fast	Moderate
Primary role	Subagents, classification	Daily coding (90%+)	Architecture, agent teams
Analogy	Intern who's quick	Senior engineer	Principal architect

The 5x Output Cost Rule

- Across all three models, output tokens cost exactly 5x input tokens
- This has real engineering implications:

  - Reading code is cheap. An agent scanning 10 files to understand your codebase costs relatively little.
  - Writing code is expensive. The implementation — every line the model generates — costs 5x more per token.
  - Design prompts to minimize unnecessary output. Point the model to existing patterns ("follow the pattern in lib/api/weather.ts") instead of asking it to generate everything from scratch.
  - Scope control matters financially. "Implement ONLY what is specified" isn't just about quality — it prevents the model from generating thousands of extra output tokens for features you didn't ask for.

How Claude Code Routes Between Models

- Claude Code uses smart model switching — it doesn't run everything on the same model
- Automatic routing:
  - Subagents (file exploration, code search, classification) → Haiku
  - Main implementation tasks → Sonnet (default) or Opus (if selected)
- User-controlled selection:
  - Default mode uses Sonnet for main tasks
  - /model command switches to Opus for complex work
  - Headless pipelines (claude -p) use whichever model is configured
- The routing principle: Use the smallest model that can handle the task reliably. Don't pay Opus prices for a file read that Haiku handles perfectly.

Cost Estimation for Real Work

A practical way to think about costs:

- A single feature implementation (Sonnet): ~40-60K tokens total, ~$0.50-1.50
- A full-day coding session (Sonnet, interactive): ~500K-1M tokens, ~$5-15
- A headless pipeline building 7 features (Sonnet): ~350K-500K tokens, ~$5-15
- A complex architecture analysis (Opus): ~100-200K tokens, ~$3-8

These costs are rough estimates — actual usage depends on codebase size, conversation length, and how many iterations the agent needs. The key insight: an afternoon of agentic engineering costs less than a lunch.

When to Upgrade to a Bigger Model

Move up the hierarchy when:

- Haiku → Sonnet: The task requires multi-step reasoning, nuanced code generation, or understanding complex relationships between files
- Sonnet → Opus: The task involves architectural decisions across a large codebase, requires agent teams for parallel work, or needs the 128K output ceiling for large-scale generation
- Rule of thumb: Start with Sonnet. If the output quality isn't meeting your standards on a specific task, try Opus. If you're running high-volume simple tasks, drop to Haiku.

What Is a Token?

[simple]

Not words, not characters — **word fragments**. "tokenization" becomes ["token", "ization"]. Roughly **1 token ~ 4 characters ~ 3/4 of a word**

Tokens are the **currency of AI** — you pay for them (money), they fill up the model's memory (space), and more of them means longer wait times (speed)

Reading is cheap, writing is expensive — the model generating code costs 5x more than the model reading your code

Think of the context window as a **whiteboard with a fixed size** — everything the model can see right now has to fit on it. System prompt, your files, the conversation, its own answers — all competing for space

Every wasted token is like writing irrelevant notes on that whiteboard — less room for what actually matters

© Simple Technology Solutions, Inc.

What Tokens Actually Are

- Tokens are not characters, not words, and not lines. They are subword units — chunks that a tokenizer algorithm decided are the most efficient way to break up text.
- Built using Byte-Pair Encoding (BPE): the algorithm starts with individual characters, then iteratively merges the most frequent pairs into single tokens. Common words like "the" or "function" become single tokens. Rare words get split: "tokenization" becomes ["token", "ization"].
- Code tokenizes differently than prose. Common programming keywords (return, const, import) are single tokens. Variable names, especially camelCase or unusual identifiers, get split into multiple tokens.
- Whitespace and indentation consume tokens. A 4-space indent is typically 1 token. This is why heavily indented code costs more than flat code.

Example: Anthropic's Claude Tokenizer in Action

Plain English — 44 characters, 9 words, 10 tokens (4.4 chars/token)

`

"The quick brown fox jumps over the lazy dog."

→ [The] [quick] [brown] [fox] [jumps] [over] [the] [lazy] [dog] [.]
1 2 3 4 5 6 7 8 9 10

`

Common English words map one-to-one to tokens. Punctuation is its own token. Leading spaces attach to the word that follows.

AI-related prose — 54 characters, 8 words, 12 tokens (4.5 chars/token)

`
"Claude Code reads your codebase and writes TypeScript."

→ [Cl] [aude] [Code] [reads] [your] [code] [base] [and] [writes] [Type] [Script] [.]
  1 2 3 4 5 6 7 8 9 10 11 12
`

Notice: "Claude" splits into Cl + aude (uncommon proper noun). "codebase" splits into code + base (compound word). "TypeScript" splits into Type + Script (camelCase). Common words like "reads," "your," and "and" stay as single tokens.

Code — 91 characters, 12 words, 26 tokens (3.5 chars/token)

`
"function calculateTotal(items) { return items.reduce((sum, item) => sum + item.price, 0); }"

→ [function] [calculate] [Total] [(] [items] [)] [{] [return] [items] [.]
  1 2 3 4 5 6 7 8 9 10
 [reduce] [((] [sum] [,] [item] [)] [=>] [sum] [+] [item] [.] [price]
  11 12 13 14 15 16 17 18 19 20 21 22
 [,] [0] [);] [}]
  23 24 25 26
`

Code uses ~3.5 characters per token vs ~4.5 for prose — syntax characters like (,), ., , each consume a full token. The keyword function is a single token, but calculateTotal splits into calculate + Total. Operators like => and + get their own tokens.

Key takeaway: Code is ~25% more expensive per character than prose because of punctuation and syntax overhead. Every bracket, dot, comma, and semicolon costs a full token.

The Three Cost Levers

1. Money (API Charges)

Model	Input Cost (per MTok)	Output Cost (per MTok)	Output Multiplier
Haiku 4.5	$1	$5	5x
Sonnet 4.6	$3	$15	5x
Opus 4.6	$5	$25	5x

- The 5x output cost is the single most important pricing fact. When an agent writes a 200-line TypeScript file, those output tokens cost 5x what it costs to

read the same file as input.
- Prompt caching saves 90% on repeated context (cached reads = 10% of input price). System prompts and CLAUDE.md benefit heavily from caching since they're identical across invocations.
- Batch API saves 50% on all costs for non-interactive workloads.
- Combined (caching + batch): up to 95% cost reduction for pipeline-style builds.

2. Space (Context Window Consumption)

- The context window has a hard token limit: 200K (Haiku), 1M (Sonnet/Opus).
- Everything competes for that space: system prompt (~1K tokens), CLAUDE.md (~2K tokens), tool definitions (~3K tokens), conversation history (grows over time), file reads (can be very large), and the model's own output.
- When the window fills up, older content gets compressed or dropped. The model loses access to earlier information.
- This is why "one agent, one prompt, one purpose" matters — a focused session uses context efficiently instead of bloating it with unrelated information.

3. Speed (Latency)

- The model generates output one token at a time, sequentially. Longer outputs take proportionally longer.
- Input tokens are processed in parallel (the "prefill" phase), so reading a large file is fast. But generating a large file is slow because each token depends on all previous tokens.
- More context = slightly slower processing due to the attention mechanism's computational cost scaling with context size.
- For interactive use, speed matters. For headless pipelines, cost usually matters more than speed.

Token Budget Mental Model

Think of your context window as a budget. Here's how a typical agent session spends it:

```
Context Window Budget (200K example)
 ├── System prompt:         ~1,000 tokens  (fixed cost)
 ├── CLAUDE.md:             ~2,000 tokens  (fixed cost)
 ├── Tool definitions:      ~3,000 tokens  (fixed cost)
 ├── User prompt:           ~500 tokens    (per-task)
 ├── File reads:            ~20,000 tokens (variable)
 ├── Conversation history:  ~30,000 tokens (grows over time)
 ├── Model's responses:     ~40,000 tokens (grows over time)
 └── Remaining budget:      ~103,500 tokens
```

- Fixed costs (system prompt, CLAUDE.md, tool definitions) are paid on every invocation. This is why keeping CLAUDE.md concise matters — a 500-line CLAUDE.md might cost 5,000+ tokens on every single call.
- Variable costs (file reads, conversation) grow during a session. This is why context degrades — as the window fills, less room remains for the model to reason effectively.
- Fresh sessions reset the variable costs to zero. This is the engineering

argument for short, focused sessions over long, bloated ones.

Practical Token Efficiency

- CLAUDE.md: Keep it under 150 lines. Every line competes for the model's attention budget. A tight CLAUDE.md with only essential conventions saves thousands of tokens per invocation.
- Prompts: Be specific and concise. "Build the weather API route following the pattern in lib/api/weather.ts" is cheaper and more effective than a 500-word description of what a weather API should do.
- File reads: Read only what you need. A focused Grep for a specific pattern is cheaper than reading an entire 2,000-line file.
- Scope control: "Implement ONLY what is specified" prevents the model from generating unnecessary output tokens for features you didn't ask for.

Why Engineers Should Care

Tokens are the fundamental unit of everything in LLM engineering:

- Cost estimation: Tokens let you estimate what a build pipeline will cost before running it. 7 features x ~50K tokens each x Sonnet pricing = predictable budget.
- Context planning: Knowing your token budget lets you design sessions that stay within the effective range instead of degrading.
- Performance tuning: Understanding the 5x output cost shifts how you design prompts — point the model to existing patterns instead of asking it to generate everything from scratch.
- Architecture decisions: Token economics drive the choice of short, focused sessions over long ones, CLAUDE.md brevity over verbosity, and verification gates over manual review.

Why Do We Have to Be Token Efficient?

[simple]

The whiteboard gets worse as it fills up — the more you cram on, the harder it is for the model to find what matters. Research shows accuracy drops below 50% in bloated contexts

Writing costs 5x more than reading — every unnecessary line of code the model generates burns the expensive tokens. "Build only what I asked for" saves real money

More tokens = slower responses — the model writes one token at a time. Ask for too much and you're waiting

Junk in your instructions compounds — a bloated CLAUDE.md wastes thousands of tokens on *every single call*. Multiply that across a pipeline and it adds up fast

The difference between "AI is too expensive" and "that cost less than lunch" is **token discipline**

© Simple Technology Solutions, Inc.

The Four Costs of Every Token

Every token you send or receive has four simultaneous costs. Understanding all four is what separates engineers who think AI is expensive from those who run it cheaply at scale.

1. Money — API charges

- Output tokens cost 5x input tokens across all Claude models
  - Haiku: $1 input / $5 output per MTok
  - Sonnet: $3 input / $15 output per MTok
  - Opus: $5 input / $25 output per MTok
- This asymmetry means the most expensive thing your agent does is write code — not read it, not analyze it, not search for files
- A model generating 500 lines of unnecessary code (dark mode, i18n, extra error handling you didn't ask for) can cost 10-20x more than a focused implementation
- Prompt caching saves 90% on repeated input (cached reads = 10% of input price). System prompts and CLAUDE.md benefit heavily because they're identical across invocations
- Batch API saves 50% on all costs for non-interactive workloads
- Combined (caching + batch): up to 95% cost reduction

2. Space — Context window consumption

- The context window has a hard limit: 200K tokens (Haiku), 1M tokens (Sonnet/Opus)
- Everything competes for that space:
  - System prompt: ~1K tokens (fixed, every call)
  - CLAUDE.md: ~1-5K tokens (fixed, every call)
  - Tool definitions: ~3K tokens (fixed, every call)
  - Conversation history: grows over time
  - File reads: variable, can be very large
  - Model's own output: grows over time
- When the window fills, older content gets compressed or dropped — the model loses access to earlier instructions and context
- Wasting space on unnecessary tokens means less room for the information that actually matters

3. Speed — Latency

- Input tokens are processed in parallel (the "prefill" phase) — reading a 10,000-token file is fast
- Output tokens are generated sequentially — each token depends on all previous tokens. Longer output = proportionally longer wait
- More context also slows processing due to the attention mechanism's computational cost, which scales with context size
- For interactive use, this is the difference between a responsive agent and one that makes you wait. For headless pipelines, cost usually matters more than speed

4. Attention — Quality degradation

- This is the cost most people miss. Every token you add doesn't just take up space — it actively degrades the model's ability to attend to everything else
- The attention mechanism has a quadratic cost: adding token N means it must attend to all N-1 previous tokens, and all previous tokens must now attend to it
- "Lost in the Middle" research: at 32K tokens of context, 11 of 12 models tested dropped below 50% of their short-context performance. The model reliably processes the beginning and end but struggles with the middle
- This means a bloated CLAUDE.md doesn't just waste space — it actively makes the model worse at following the instructions that matter

Why CLAUDE.md Brevity Compounds

CLAUDE.md is loaded on every single invocation. This makes it the highest-leverage place to be token efficient.

`
Scenario A: 500-line CLAUDE.md (~5,000 tokens)
  - 7 feature builds × 5,000 tokens = 35,000 input tokens wasted
  - At Sonnet pricing: 35,000 × $3/MTok = $0.11 per pipeline run
  - Plus: attention diluted across 5,000 tokens of instructions
  - Result: model misses critical conventions buried in the noise

Scenario B: 80-line CLAUDE.md (~800 tokens)
  - 7 feature builds × 800 tokens = 5,600 input tokens
  - At Sonnet pricing: 5,600 × $3/MTok = $0.02 per pipeline run
  - Plus: model focuses attention on only the critical conventions

- Result: higher compliance with fewer, clearer instructions
`

The dollar savings are modest on a single run. The quality difference is what matters — every unnecessary line in CLAUDE.md is a line competing for the model's attention against your critical instructions.

Scope Control as Token Efficiency

When a model generates code you didn't ask for, it wastes tokens in three ways:

1. Output tokens — The unnecessary code itself costs 5x input rates
2. Context consumption — That code now sits in the conversation, displacing useful information
3. Downstream confusion — Future turns have to reason about code that shouldn't exist

This is why "Implement ONLY what is specified" and "Do not add features beyond these requirements" in build prompts aren't just quality instructions — they're cost optimization.

Common unnecessary output that wastes tokens:
- Adding dark mode, i18n, or accessibility features not in the spec
- Generating verbose comments and JSDoc for every function
- Creating error handling for scenarios that can't happen
- Building configuration options and feature flags for a prototype
- Writing tests when you didn't ask for tests

Fresh Context as Token Efficiency

Starting a fresh session is the most aggressive form of token efficiency:

- Conversation history: Reset to zero. No accumulated noise from previous turns
- Tool results: Reset to zero. No lingering file reads from earlier exploration
- Model output: Reset to zero. No previous implementations cluttering context
- Only fixed costs remain: System prompt + CLAUDE.md + tool definitions

This is why short, focused sessions produce better results than long, bloated ones. It's not just that context degrades — it's that a fresh session gives the model maximum space and attention for the task at hand.

The Compound Effect

Token efficiency compounds across every dimension:

Practice	Money Saved	Quality Gained	Speed Gained
Tight CLAUDE.md (80 vs 500 lines)	~80% fewer fixed input tokens	More attention on critical rules	Faster context loading
Scope control in prompts	Fewer unnecessary output tokens	Focused implementation	Less generation time
Fresh context per task	No wasted history tokens	Full attention on current task	No bloated prefill

| Pointed file reads (Grep vs full Read) | Fewer input tokens per lookup | Less noise in context | Faster tool results |
| "Follow pattern in X" vs "generate from scratch" | Fewer output tokens | Consistent with existing code | Less generation time |

An engineer who practices token efficiency across all five dimensions can run the same workload at a fraction of the cost with higher quality output. The difference between "AI is expensive and unreliable" and "AI costs less than lunch and follows my conventions" is often just token discipline.

What Is an Agent?

[simple]

A model by itself is a **brain in a jar** — it can think, but it can't do anything. An agent is that brain with hands, eyes, and a terminal

The agent loop: **think → do something → look at what happened → think again → repeat** until the job is done

Claude Code is an agent — it reads your files, writes code, runs the build, sees the error, fixes it, and tries again. That loop is the whole game

Every agent framework (Claude Code, Cursor, Copilot, Codex) converges on the **same 6 lines of logic** — a while loop that calls a model, checks for tool calls, runs them, and loops

One-shot answers are guesses. **The loop is what makes it reliable** — the agent can recover from mistakes, iterate, and verify its own work

© Simple Technology Solutions, Inc.

Model vs. Agent — The Core Distinction

Model (one-shot):
- Text in → text out → done
- A stateless function call. No memory of what happened, no ability to take action, no way to verify its output
- Like asking a colleague a question over email — you get one response, and if it's wrong, you have to write another email

Agent (loop):
- Model + tools + context + loop
- A running process that persists, decides, acts, observes, and iterates
- Like having that colleague sit next to you with access to your codebase, terminal, and browser — they can try things, see what happens, and keep going until it works

The distinction is not about intelligence. The same model powers both. The difference is the architecture around it.

The Agent Loop

Every agent follows the same four-step cycle:

`

```
┌─→ THINK    — The model reasons about the current state
│    ↓
│   ACT      — Call a tool (read a file, run a command, write code)
│    ↓
│   OBSERVE  — Examine the tool's result (build output, error message, file
contents)
│    ↓
└── REPEAT   — Loop back to THINK with the new information
       ↓
     DONE   — No more tool calls needed, return the final response
```

This is the ReAct (Reasoning + Acting) pattern. The model doesn't just reason about what to do — it actually does it, sees the result, and adjusts.

The Canonical Agent Loop in Code

Every major agent framework converges on this same structure:

```python
def agent_loop(llm):
    msg = user_input()
    while True:
        output, tool_calls = llm(msg)
        if tool_calls:
            msg = [execute(tc) for tc in tool_calls]
        else:
            return output
```

Six lines. That's the core of Claude Code, Cursor, Codex, LangGraph, Vercel AI SDK — all of them. Steve Kinney (Temporal) called them "the same six lines of logic wearing different costumes."

The engineering challenge isn't the loop itself — it's everything around it:
- Context management — what information goes into the model at each step
- Tool design — what actions the agent can take and how results feed back
- Security — what the agent is allowed to do (permission levels, sandboxing)
- Cost control — how many iterations to allow before stopping
- Verification — how to confirm the agent's work is correct

Why Agents Beat One-Shot Calls

Error recovery:
- A one-shot model call that generates broken code just gives you broken code. You have to manually read the error, figure out the fix, and prompt again.
- An agent generates code, runs the build, reads the error, reasons about the fix, applies it, and runs the build again — automatically.

Task decomposition:
- Complex tasks rarely succeed in a single generation. The agent naturally breaks work into steps: read the existing code first, plan the approach, implement piece by piece, verify at each step.

Self-verification:

- The agent can run tsc --noEmit, npm run build, or a test suite and check its own work. The model's self-assessment ("I think this is correct") is unreliable. The compiler's assessment is deterministic.

Research shows the difference:
- The ReAct framework demonstrated 34% improvement on complex tasks compared to one-shot reasoning.
- Multi-step allows the agent to gather information it needs (reading files, checking types, inspecting errors) before committing to an implementation.

Claude Code as an Agent — Concrete Example

When you give Claude Code a task like "add a search feature to the dashboard," here's what actually happens:

1. THINK — Reasons about the task. Decides it needs to understand the existing dashboard structure first.
2. ACT — Calls Glob to find dashboard-related files. Calls Read to examine the main dashboard component.
3. OBSERVE — Sees the component structure, state management pattern, existing imports.
4. THINK — Plans the search implementation: new component, filter logic, integration with existing state.
5. ACT — Calls Write to create the search component. Calls Edit to add it to the dashboard layout.
6. OBSERVE — Files written successfully.
7. ACT — Calls Bash to run tsc --noEmit.
8. OBSERVE — TypeScript error: missing type for the search prop.
9. THINK — Reads the error, identifies the missing type definition.
10. ACT — Calls Edit to add the type. Calls Bash to run tsc --noEmit again.
11. OBSERVE — Clean compilation. Calls Bash to run npm run build.
12. OBSERVE — Build passes.
13. DONE — Returns summary of what was built.

That's 5-6 iterations of the loop for a single feature. Each iteration gathers information or takes action that the previous iteration couldn't. A one-shot call would have to get all of that right on the first try — and it almost never would.

Claude Code's Tool Set

The tools are what give the agent its "hands":

Tool	What It Does	Analogy
Read	Inspect file contents	Opening a file in your editor
Write	Create new files	Saving a new file
Edit	Targeted string replacements	Find-and-replace in your editor
Bash	Execute shell commands	Typing in your terminal
Glob	Find files by pattern	find or file explorer search
Grep	Search file contents	grep or IDE search
Agent	Spawn sub-agents	Delegating a subtask to a teammate

Each tool call generates a result that feeds back into context. The model sees exactly what you would see if you ran the same command — the output of npm run build, the contents of a file, the list of matching files.

The "While Loop" Mental Model

For the audience of software engineers, the simplest mental model is:

- An agent is a while loop that calls an LLM on each iteration
- The LLM can return either a final answer (break the loop) or a tool call (continue the loop)
- Tool calls are just function invocations — the LLM specifies the function name and arguments as JSON, the runtime executes them, and the result goes back to the LLM
- The loop continues until the LLM decides it's done or a safety limit is hit

This is familiar territory for any engineer who's built a REPL, a game loop, or an event loop. The only novel element is that the "controller" deciding what happens next is an LLM instead of hardcoded logic.

What Is the Context Window?

[simple]

The model's **whiteboard** — everything it can see right now. System prompt, your files, the conversation, its own answers, all sharing one fixed-size surface

When the whiteboard fills up, **it starts erasing the middle**. Research shows accuracy drops below 50% in bloated contexts — a cluttered whiteboard is worse than a big one

~120K tokens is your danger zone. Once you cross that threshold, start thinking about compacting, clearing, or starting a fresh agent — don't wait until quality visibly degrades

`/compact` is Claude's built-in eraser — it summarizes older conversation to free up space. Handy, but it's a lossy summary — sometimes Claude loses the plot after a compact and forgets what it was doing

Fresh whiteboard = fresh attention. Starting a new session resets everything. 11 short focused sessions beat one long degrading marathon. When in doubt, start fresh instead of compacting

New task? New agent. When you shift to a different problem, clear the whiteboard and start fresh — don't let leftover context from the last task pollute the next one

© Simple Technology Solutions, Inc.

What's Actually on the Whiteboard

The context window holds everything the model can "see" at inference time. It all competes for the same fixed space:

```
Context Window (e.g., 1M tokens for Sonnet/Opus)
 ├── System prompt        ~1,000 tokens   (Claude Code's built-in instructions)
 ├── CLAUDE.md            ~1,000-3,000    (your project conventions)
 ├── Tool definitions     ~3,000          (Read, Write, Edit, Bash, etc.)
 ├── Conversation history  grows over time (every message back and forth)
 ├── Tool results         grows over time (file contents, build output, grep
results)
 ├── Model's own output    grows over time (every response it's generated)
 └── Remaining capacity    shrinks over time
```

Early in a session, most of the whiteboard is empty — the model has maximum room to work. As the conversation continues, history and tool results accumulate, and the remaining capacity shrinks.

The ~120K Token Threshold

The models technically support massive context windows (200K–1M tokens),

but the usable sweet spot is much smaller. Around ~120K tokens, you should start actively managing context:

- Below 120K — The model is generally sharp. Attention is focused, instructions are followed well, code quality is consistent.
- 120K–200K — Degradation starts creeping in. The model may miss conventions, repeat itself, or lose track of earlier context. This is your signal to compact, clear, or start fresh.
- Above 200K — Quality drops noticeably even on models that support 1M. The "Lost in the Middle" effect is in full force. You're fighting the architecture instead of using it.

How fast do you hit 120K?
- A single large file read: 5–10K tokens
- A verbose conversation with 20 back-and-forth turns: 50–100K tokens
- Tool results (build output, grep results, file contents) accumulate fast
- The model's own responses are often the biggest consumer — output is verbose
- A focused coding session can hit 120K in 30–60 minutes of active work

What to do when you're approaching the threshold:
1. Compact — Use /compact in Claude Code to summarize older conversation
2. Clear — Use /clear to reset the conversation entirely
3. Start fresh — Open a new session. CLAUDE.md reloads, the filesystem persists, nothing is lost
4. Isolate — Delegate subtasks to subagents that get their own clean context

The key insight: don't wait until you notice quality dropping. By the time you notice, you've already wasted several turns of degraded output. Treat 120K as a proactive checkpoint, not a reactive alarm.

Context Degradation: Why It Happens

The "Lost in the Middle" Problem:
- Research by Liu et al. (2023) tested 12 language models on information retrieval tasks at various context lengths
- At 32K tokens, 11 of 12 models dropped below 50% of their short-context performance
- Models reliably process the beginning (system prompt, CLAUDE.md) and the end (recent messages) but struggle with the middle (older conversation, earlier tool results)
- This is why critical instructions belong in CLAUDE.md (loaded at the start) or in the current prompt (at the end) — not buried in conversation history

Quadratic Attention Cost:
- The transformer attention mechanism has $O(n^2)$ complexity with respect to context length
- Every token added doesn't just take up space — it requires every other token to attend to it
- At 100K tokens, adding 1K more tokens creates 100 million new attention computations
- This isn't just a performance cost — it dilutes the model's attention across more information, making it harder to focus on what matters

Symptoms of Context Degradation:
- The model starts contradicting earlier decisions

- It forgets conventions established in CLAUDE.md
- Responses become verbose and hedging ("I think maybe we could consider...")
- It re-reads files it already read earlier in the session
- Code quality drops — more bugs, less consistency with established patterns

Fresh Context as a Strategy

Starting a new session (claude -p or a new conversation) resets the context window completely:

- What's preserved: Files on disk, git history, CLAUDE.md — the filesystem is the persistent memory
- What's lost: Conversation history, previous reasoning, tool results from earlier turns
- What's gained: Full attention capacity, no degradation, clean whiteboard

This is counterintuitive — it feels wasteful to "forget" previous work. But the model doesn't need to remember building Feature 1 to build Feature 2. It just needs to read the code Feature 1 left behind on disk. The filesystem is the memory; the context window is the working scratch space.

New Task, New Agent — Clear the Whiteboard Between Concepts

The most common mistake is treating one agent session as a general-purpose workhorse for an entire workday. Context left over from debugging a CSS issue will pollute the agent's attention when you pivot to designing an API schema.

The rule: when you change what you're thinking about, start a fresh agent.

- Finished debugging? Start a new session before implementing the next feature.
- Done with research? Start a new session to begin coding.
- Switching from frontend to backend work? Fresh session.
- About to do something complex? Don't fight through a degraded whiteboard — restart.

In practice:
- In Claude Code interactive: use /clear to reset the conversation, or close and reopen
- In headless mode: each claude -p call is already a fresh session by design
- In claude.ai: start a new chat instead of continuing a long thread

Why this works:
- CLAUDE.md reloads automatically on every fresh session — your conventions carry forward
- The codebase is on disk — the agent re-reads what it needs from the filesystem
- Git history preserves everything — no work is lost by resetting context
- The only thing you "lose" is conversation history, and that's usually noise by the time you're changing topics

The analogy: You wouldn't keep yesterday's meeting notes on the whiteboard when starting today's design session. You'd erase the board and start clean. Same principle.

The Four Context Management Strategies

From Anthropic's "Effective Context Engineering for AI Agents" (Sep 2025):

1. Write — Externalize to persistent storage
- Save intermediate results to files instead of keeping them in context
- Use scratchpads, todo lists, or summary files that the agent can read back later
- Example: Agent writes a plan to plan.md, then reads it back in a fresh session

2. Select — Only load what's relevant
- Use targeted file reads (Grep for a pattern) instead of reading entire files
- Search for specific functions instead of loading whole codebases
- Example: Grep("handleSubmit") instead of Read("src/app.tsx") when you only need one function

3. Compact — Summarize older context
- Claude Code's automatic context compaction summarizes older conversation when the window fills
- Older messages get condensed while recent messages retain full detail
- This happens automatically but still degrades quality vs. a fresh session

4. Isolate — Give subtasks their own context
- Spawn subagents (via the Agent tool) that get their own clean context window
- The subagent does focused work and returns a concise result to the parent
- Example: Main agent delegates "find all files that import UserContext" to a Haiku subagent — the search results fill the subagent's context, not the parent's

The Computer Architecture Analogy

For a software engineering audience, the most intuitive mental model:

Computer Component	LLM Equivalent
CPU	The model itself (reasoning engine)
RAM	Context window (working memory — fast, limited, volatile)
Hard drive	Training data / parametric knowledge (large, slow, persistent)
I/O ports	Tools (Read, Write, Bash — how the model interacts with the world)
Swap space	Context compaction (when RAM fills, older data gets compacted)

Just like a computer slows down when RAM is full and starts swapping to disk, a model degrades when context is full and starts compacting older conversation. The solution is the same: manage your memory carefully, and restart when things get sluggish.

Why Do We Use Markdown?

[simple]

It's the **one format both you and the AI read natively** — no parsing, no conversion, no special tools. You write it, the model reads it, done

Markdown is **structured enough to convey priority** — headers signal sections, bold signals importance, code blocks signal "use this exactly." The model picks up on all of it

It's **diffable, committable, and version-controlled** — your prompts, specs, and research live in git alongside your code, not lost in a chat window

Every step of the process is a markdown file: spec, research, CLAUDE.md, build prompts — **markdown in, software out**

If you can write a good README, you can drive an agent. There's no special "prompt language" — **it's just clear writing**

© Simple Technology Solutions, Inc.

Markdown as the Universal Interchange Format

In agentic engineering, information moves constantly between humans and models — specs, instructions, research results, code context, build prompts. Markdown works as the interchange format because it satisfies both sides:

For humans:
- Readable in any text editor, GitHub, VS Code preview, or just cat in a terminal
- Familiar to every software engineer — READMEs, docs, wikis, PR descriptions
- Easy to write quickly without tooling

For models:
- Parsed natively during training — models have seen millions of markdown documents
- Structure maps to attention priority (more on this below)
- No special tokens or formatting needed — just text

How Markdown Structure Maps to Model Behavior

Models don't treat all text equally. Markdown formatting carries semantic weight:

Markdown Element	How the Model Interprets It

# Heading	Major section boundary — resets context focus
bold text	Higher priority instruction — gets more attention weight
- bullet point	Discrete item in a list — treated as a separate instruction
` code block `	Exact syntax — should be used literally, not paraphrased
> blockquote	Referenced/quoted material — distinct from direct instructions
--- (horizontal rule)	Section separator — signals a context boundary

This is why CLAUDE.md files use bold for critical conventions and code blocks for exact commands — the formatting isn't decorative, it's functional. A bold "Do NOT use tailwind.config.js" gets more attention than a plain sentence.

Why Not JSON, YAML, or XML?

- JSON — Great for machine interchange, painful for humans to read and write. A 50-line spec in JSON is 150 lines with braces and quotes.
- YAML — Readable but whitespace-sensitive in ways that cause subtle bugs. Models sometimes generate invalid YAML.
- XML — Verbose, hard to read, overkill for natural-language instructions.
- Plain text — No structure. The model can't distinguish a heading from a detail, a critical instruction from a casual note.
- Markdown — Structured enough to convey hierarchy and priority, lightweight enough to write in seconds, readable by both humans and machines without any tooling.

Markdown in the Agentic Workflow

Every stage of agentic engineering uses markdown:

Configuration:
- CLAUDE.md — Project conventions, build commands, critical rules
- System prompts — Persistent instructions for agent behavior

Research:
- Research prompts — Structured questions fed into claude.ai
- Research results — Synthesized findings saved as .md files

Specification:
- Feature specs — What to build, API details, acceptance criteria
- Tech stack decisions — What was chosen and why

Build:
- Build prompts — Per-feature specs with "patterns to follow" sections
- Build scripts — Reference markdown files via cat build/prompts/XX.md

Communication:
- PR descriptions, commit messages, documentation — all markdown

The "Prompt Engineering" Demystification

There's a mystique around "prompt engineering" — as if there's some secret language for talking to AI. With markdown, the skill is just clear technical writing:

- State what you want built (heading + bullets)
- Provide the context needed (attached markdown files)

- Specify constraints ("implement ONLY what is specified")
- Point to patterns ("see lib/api/weather.ts for the pattern to follow")

If you've ever written a good ticket, a clear README, or a detailed code review — you already have the skill. The format is markdown. The craft is clarity.

Why It Beats Chat Windows

Prompts and research typed into a chat window are:
- Ephemeral — lost when the session ends or the window closes
- Not diffable — you can't see what changed between version 1 and version 2
- Not shareable — a teammate can't re-run your exact prompt
- Not version-controlled — no git history, no blame, no rollback

Markdown files in a repo are the opposite of all of those. The entire reasoning chain — from initial research to final build — is transparent, repeatable, and auditable as plain text in git.

What Is a Markdown Refinement Chain?

[simple]

Have Claude write a prompt, paste it into claude.ai with web search, save the result as a .md file in your repo — that's the whole system

Each markdown file feeds the next one — company briefing → demo spec → tech stack → build prompts → working software. Every step refines the last

Use the right tool for the job — claude.ai can search the web and read live docs. Claude Code can write code and run builds. Don't make one do the other's job

Research gets its own whiteboard — a dedicated session means the model's full attention is on research, not split with your codebase and conversation history

Everything is in the repo — prompts, research results, specs, build prompts. The whole chain is transparent, repeatable, and version-controlled

© Simple Technology Solutions, Inc.

The Core Idea: Separate Research from Implementation

A markdown refinement chain is a pipeline where each step produces a markdown file that becomes the input to the next step. You tell Claude what you need, Claude writes it, you review and refine, then feed it forward. It's a deliberate separation of two fundamentally different tasks:

- Research — browsing the web, reading documentation, synthesizing information from multiple sources, finding current best practices. Needs: web access, large context for synthesis, deep reasoning.
- Implementation — writing code, editing files, running builds, debugging. Needs: terminal access, file system tools, compiler feedback loops.

Claude.ai and Claude Code are optimized for different halves of this work. Trying to do both in one session wastes the model's attention on the wrong things.

Why claude.ai for Research?

Web search and live browsing

- Claude.ai with "Research" and "Web Search" enabled can search the internet, follow links, read documentation sites, and synthesize findings from multiple sources in real time.

- Claude Code operates on your local filesystem and terminal. It can read files and run commands, but it doesn't natively browse the web the same way.
- For a task like "find me the best conference talks on agentic engineering from 2025-2026," you need a model that can actually go look.

Opus 4.6 Extended thinking

- The "Extended" option in claude.ai gives the model more reasoning capacity for complex synthesis tasks.
- Research isn't just finding links — it's reading 10 sources and producing a coherent synthesis. Extended thinking gives the model room to reason through contradictions, weigh sources, and organize a large body of information.

Full context dedicated to one task

- A research session that reads 15 web pages and synthesizes them into a structured document can easily consume 100K+ tokens.
- If you tried to do this inside a Claude Code session that's also holding your codebase context, CLAUDE.md, and conversation history, you'd be competing for whiteboard space.
- A dedicated claude.ai session gives the research task the model's full attention and full context window.

Why Store the Result as a Markdown File?

It becomes a link in the refinement chain

- The research prompt system is one step in a larger pipeline where each markdown artifact produces the next:
  1. Research prompt (.md) → fed into claude.ai
  2. Research result (.md) → informs the next prompt
  3. Spec or decision record (.md) → informs build prompts
  4. Build prompts (.md) → fed into Claude Code
  5. Working software → verified and committed
- Each file is both an output of one step and an input to the next. The chain is only as good as each link.

It's reusable and improvable

- The research result stays in the repo as a reference document. When you need to make decisions later, the research is there — you don't have to redo it.
- If the research becomes stale (new frameworks, updated best practices), you re-run the prompt and get a fresh result. The prompt itself is the repeatable recipe.

It's transparent and auditable

- Anyone on the team can read the research prompt to understand what was asked and read the result to understand what was found.
- There's no hidden context or lost conversation. The entire reasoning chain is committed to the repo as plain text.

Why the Research Prompt Lives in the Repo

Prompts are engineering artifacts

- A well-crafted research prompt is like a well-written test — it encodes your intent precisely enough that a machine can execute it and produce a reliable result. And Claude writes it for you — you describe what you need, review the prompt, ask for changes, and approve it.
- Committing it to the repo means it's version-controlled, diffable, and reviewable. You can see how the prompt evolved and why.

Anyone can re-run or adapt it

- A teammate can copy the prompt into their own claude.ai session and get comparable results.
- For a different topic, they can adapt the prompt's structure (the "what I already have," "what I need," "what I want from this research" sections) without starting from scratch.

It separates "what to ask" from "what was found"

- The prompt file and the result file are separate artifacts. This matters because:
  - The prompt captures your intent and constraints
  - The result captures the model's findings at a point in time
  - You can re-run the same prompt months later and compare results to see what's changed

The Refinement Chain Pattern

This research prompt system is one instance of a broader pattern: markdown refinement chains.

```
Idea (in your head)
  → Research prompt (.md)       — Claude writes this, you review
    → Research result (.md)     — claude.ai produces this
      → Spec / decision (.md)   — Claude writes this, you review
        → Build prompts (.md)   — Claude writes these, you review
          → Working software    — Claude Code produces this
```

At each step:
- You tell Claude what you need — Claude writes the markdown document
- You review the result — maybe ask Claude to revise it, maybe it's good as-is
- That document becomes the input to the next step

You're not writing any of it. You're directing, reviewing, and deciding when each artifact is good enough to feed into the next step. The human role is judgment and intent, not authorship.

This is the "building the system that builds the system" principle in action. You're not writing code, and you're not even writing the documents — you're directing the AI that writes the documents that produce the documents that produce the code.

[simple]

ACT 2

How I Built a Demo App

Watch the concepts in action — here's what it actually looks like.

© Simple Technology Solutions, Inc.

Step 1 — Who Am I Presenting To?

[simple]

Before writing a line of code, I needed to understand my audience — **have Claude research the company first**

One prompt into claude.ai with web search: "write a summary about this company, I'm presenting a brown bag and want context about the people I'll be presenting to"

Result: a markdown briefing on STS — $60M DOI contract, 3,000 officers, 855+ locations, AI products (C-CAT, LEI, A3), "Keep IT Simple" culture

This briefing **becomes the input to the next step** — it's how Claude knows what kind of demo app to build

Total time: ~5 minutes of prompting, zero manual research

© Simple Technology Solutions, Inc.

The Actual Prompt

`
write a summary about this company, Simple Technology Solutions Inc.,
I'm presenting a brown bag to the company and want to have context
about the people I'll be presenting to. https://www.simpletechnology.io/
`

- Tool: claude.ai (Opus 4.6 Extended + Research + Web Search)
- Claude browsed the STS website, read about the company, and produced a structured briefing
- Result saved to: research/simple-technology-solutions-a-pre-presentation-briefing-on-federal-it-and-ai-capabilities.md

What the Research Produced

Key facts Claude surfaced that shaped everything downstream:

- STS is a federal IT and AI company — not a startup, not consumer tech. The audience cares about mission, compliance, and operational reliability.
- $60M DOI contract — Department of the Interior, law enforcement focus. 3,000+ officers across 855+ locations on federal lands.
- Existing AI products — C-CAT (case analysis), LEI (law enforcement intelligence), A3 (AI accelerator). The audience already understands AI has

practical applications.
- Culture: "Keep IT Simple" — practical, mission-first engineering. The demo should be useful, not flashy.

Why This Step Matters

- Without this briefing, Claude would have suggested a generic demo app (todo list, weather widget, chat app)
- With this briefing, Claude could reason about what would resonate: federal lands, law enforcement operations, real-time situational awareness
- The briefing is a markdown file in the repo — it feeds directly into the next prompt as attached context
- This is the refinement chain in action: research artifact → input to next step

Prompt Design Principles at Play

- Give Claude a URL — let it do the reading, not you
- State your purpose — "I'm presenting a brown bag" tells Claude what kind of summary to write
- Save the result as markdown — it becomes a reusable, committable artifact

Step 2 — What Free Data Can I Use?

[simple]

Next link in the chain: **find 100 free APIs and datasets** that could power a demo app built with Claude Code

One prompt: "find or create a list of 100 or so of the best open APIs or datasets that can be used to create an example application"

Claude searched the web and came back with a categorized list — weather, geospatial, government, science, finance, and more

This list becomes **attached context for the next prompt** — Claude uses it to pick which APIs fit the audience

I didn't evaluate 100 APIs manually. Claude did the research, I reviewed the list

© Simple Technology Solutions, Inc.

The Actual Prompt

`
find or create a list of 100 or so of the best open APIs or datasets
that can be used to create an example application that I will build
using claude code to demonstrate the power of claude code.
`

- Tool: claude.ai (Opus 4.6 Extended + Research + Web Search)
- Result saved to: research/100-free-apis-and-datasets-every-developer-should-know.md

What the Research Produced

A categorized list of free APIs, including the ones that ended up in the final app:

API	Category	Auth Required
Open-Meteo	Weather & forecasts	None
USGS Earthquake Hazards	Geospatial / Science	None
NASA FIRMS	Wildfire detection	Free API key
FBI Crime Data Explorer	Government	Free API key
Nominatim (OpenStreetMap)	Geocoding	None
Open-Meteo Astronomy	Science	None

Plus dozens more across categories: finance (Alpha Vantage, CoinGecko), social (Reddit, Hacker News), transportation (OpenSky), space (NASA APOD, SpaceX), government (data.gov, Census), etc.

Why This Step Matters

- The demo app needs real, working data sources — not mocked endpoints
- Free APIs with no credit card mean the demo works for anyone who clones the repo
- Having a large list gives Claude options when designing the app spec — it can match APIs to the audience
- This markdown file gets attached to the next prompt alongside the STS briefing

The Chain So Far

```
STS Briefing (.md) ─┐
                    ├─→ Next step: design the app spec
100 Free APIs (.md) ─┘
```

Two research artifacts, both produced by Claude, both feeding forward as context for the next decision.

Step 3 — What Should the App Do?

[simple]

Fed both previous markdown files into the next prompt: "make an opinionated choice about what this application should do"

Claude combined the STS briefing + API list and designed **TRAILHEAD** — a Federal Lands Daily Briefing Tool for DOI law enforcement

7 features, 6 free APIs, purpose-built for the audience — not a generic demo

Key instruction: **"be technology agnostic, only focus on features, do not include any implementation details"** — keep the spec clean

Result: a complete feature spec in markdown, ready to hand to the next step

© Simple Technology Solutions, Inc.

The Actual Prompt

```markdown
I want to create an example web application using claude code that I
present in a brown bag to STS. This application should match the
complexity of what claude code can do in 12 hours of human time
(the amount of time I have to prepare for the brown bag).
make an opinionated choice about what this application should do
- Consider the company being presented to using this briefing:
  - simple-technology-solutions-a-pre-presentation-briefing...md
- consider the list of free apis and datasets:
  - 100-free-apis-and-datasets-every-developer-should-know.md
- be technology agnostic
  - only focus on features
  - do not include any implementation details
- write the specifications in markdown
```

- Tool: claude.ai (Opus 4.6 Extended + Research + Web Search)
- Both previous research files attached as context
- Result saved to: research/sts-demo-spec.md

What Claude Designed

TRAILHEAD — Federal Lands Daily Briefing Tool

Claude chose this because the STS briefing revealed: DOI contract, 3,000 officers, 855+ federal land locations. A situational awareness tool maps perfectly to that audience.

7 Features:
- F1: Federal Lands Location Selector — curated list organized by agency, persists across sessions
- F2: Real-Time Conditions Dashboard — weather, seismic, fires, astronomy on one screen with auto-refresh
- F3: AI-Generated Daily Briefing — gathers all data, sends to Claude API, streams a 5-section law enforcement briefing
- F4: Interactive Situational Map — Leaflet map with fire/earthquake markers and station pin
- F5: Briefing History — localStorage persistence, reverse chronological browsing
- F6: Multi-Location Watch List — save multiple locations with compact alert cards
- F7: Shareable Briefing Export — copy-to-clipboard and print-friendly view

Prompt Design Principles at Play

- "Make an opinionated choice" — forces Claude to commit to a direction instead of listing 5 options. You want a decision, not a menu.
- "Be technology agnostic, only focus on features" — separating what from how keeps the spec clean. Tech stack is a separate decision with its own prompt.
- "Match the complexity of what claude code can do in 12 hours" — gives Claude a constraint to scope against. Without this, it might design something too ambitious or too simple.
- Attaching both previous files — Claude can reason across them. It matched federal lands (from the STS briefing) with geospatial/weather/seismic APIs (from the API list).

The Chain So Far

```
STS Briefing (.md) ─┐
                    ├─→ App Spec (.md) ─→ Next step: tech stack
100 Free APIs (.md) ─┘
```

Step 4 — What Tech Stack Should I Use?

[simple]

Attached the app spec and asked Claude: **"make an opinionated choice about what tech stack should be"**

Claude produced a full decision record — not just "use React" but *why* Next.js App Router, *why* Zustand over Redux, *why* TanStack Query

Every choice documented with rationale: what was picked, what was rejected, and why

This becomes the foundation for CLAUDE.md — the conventions file that guides every build step

Still zero code written. Four markdown files deep into the refinement chain

© Simple Technology Solutions, Inc.

The Actual Prompt

`markdown
I want to use react and spec driven development to build out this demo application **make an opinionated choice about what tech stack should be**
- consider the functionality being requested to build
  - sts-demo-spec.md
`

- Tool: claude.ai (Opus 4.6 Extended + Research + Web Search)
- App spec attached as context
- Result saved to: research/trailhead-tech-stack.md

The Tech Stack Decision Record

Decision	Choice	Rationale
Framework	Next.js 15 (App Router)	Server components by default, API routes built in, Vercel deploy
Language	TypeScript strict	Type safety catches agent errors at compile time
Styling	Tailwind CSS v4 + shadcn/ui	Utility-first, component library, fast iteration

Client state	Zustand	Minimal boilerplate vs Redux, good for location/watchlist state
Server state	TanStack Query v5	Auto-refresh intervals, caching, loading/error states built in
Map	React Leaflet + OpenStreetMap	Free, no API key, good React integration
AI	Claude API via Anthropic SDK	Streaming support, matches the presentation topic
Package manager	pnpm	Fast, disk-efficient
Deploy	Vercel	Zero-config Next.js deploy

What Was Deliberately Left Out

The tech stack record also documented what was excluded and why:
- Database — localStorage is sufficient for a demo. No persistence complexity.
- Authentication — unnecessary for a demo app. Would add scope without value.
- Testing — deprioritized for time. Verification gates (tsc + build) are the quality check.
- CI/CD — deploy manually via Vercel CLI. No pipeline needed for a one-afternoon build.

Why a Decision Record Matters for Agentic Builds

- When Claude Code builds the app, it needs to know why choices were made, not just what was chosen
- "Use Zustand" is less useful than "Use Zustand because we need minimal boilerplate for location and watchlist state"
- The rationale prevents the agent from second-guessing decisions or introducing alternatives mid-build
- This document feeds directly into CLAUDE.md — the conventions get distilled into the 50 lines that matter

The Chain So Far

```
STS Briefing (.md) ─┐
                    ├─→ App Spec (.md) ─→ Tech Stack (.md) ─→ Next step: repo setup
100 Free APIs (.md) ─┘
```

Step 5 — Bootstrap the Repo with One Prompt

[simple]

Wrote a **master setup prompt** (`ai-brown-bag.md`) and ran it in Claude Code — it bootstrapped the entire repo in one shot

Created: git repo, GitHub remote, CLAUDE.md, prompt capture system, memory files, downloaded Claude Code docs

CLAUDE.md is the key artifact — 50 lines of conventions distilled from the tech stack decision record. It's loaded on every future agent call

This is where **claude.ai hands off to Claude Code** — research phase is done, build phase begins

One prompt, 6 tasks completed, initial commit pushed to main

© Simple Technology Solutions, Inc.

The Actual Prompt

The master prompt (ai-brown-bag.md) instructed Claude Code to:

1. Initialize git repo with main branch, create CLAUDE.md and .gitignore
2. Create GitHub repo under the keepitsts organization
3. Design and implement a prompt capture system (prompts/ directory with sequential numbering)
4. Download latest Claude Code documentation (75 markdown files)
5. Capture all pre-existing research actions as prompt files (prompts 001–006)
6. Set up memory system for user preferences and project context

What Claude Code Produced

- CLAUDE.md — The conventions file, distilled from the tech stack decision record:

```
# TRAILHEAD — Federal Lands Daily Briefing Tool
## Tech Stack
- Next.js 15 App Router, TypeScript strict
- Tailwind CSS v4 (CSS-based config, NOT tailwind.config.js)
- shadcn/ui, Zustand for client state, TanStack Query v5
## Critical Conventions
- Server Components by default. 'use client' ONLY for interactivity
```

```
- Next.js 15: params are Promises — must await
- React Leaflet REQUIRES dynamic import with ssr: false
- Data flow: lib/api/ → app/api/ route → TanStack Query hook → component
## Verification (run after every feature)
npx tsc --noEmit && npm run build
`
```

- prompts/ — Sequential prompt capture system (001–007 at this point)
- GitHub repo — github.com/keepitsts/claude-ai-brown-bag (public)
- Claude Code docs — 75 markdown files fetched for reference

Why CLAUDE.md Is the Most Important File

- Every future claude -p call loads CLAUDE.md automatically
- It's how conventions survive across fresh context windows — each build step starts from zero memory but reads this file first
- Under 80 lines — every line competes for the model's attention (the whiteboard is finite)
- The data flow pattern (lib/api/ → route → hook → component) kept all 11 build steps consistent
- Critical "gotchas" are encoded: Tailwind v4 uses CSS config not JS, Next.js 15 params are Promises, Leaflet needs ssr: false

The Handoff Point

This is where the workflow shifts:

Phase	Tool	Purpose
Steps 1–4	claude.ai (web search)	Research and decisions — markdown artifacts
Step 5 onward	Claude Code (terminal)	Implementation — code, builds, deploys

The research artifacts (STS briefing, API list, app spec, tech stack) informed CLAUDE.md. From here forward, CLAUDE.md carries the knowledge.

The Chain So Far

```
STS Briefing → App Spec → Tech Stack → CLAUDE.md + Repo
                                  ↓
                            Ready to build
`
```

Step 6 — How Do I Make Claude Code Build It While I Walk My Dog?

[simple]

The big question: "I want to give the spec to Claude Code and walk my dog Milo. Come back, and it's done."

I knew it wouldn't work in one shot — **context fills up by feature 4, it forgets patterns, mistakes compound**

Created a research prompt asking Claude to evaluate approaches: single session, headless pipeline, Agent SDK, agent teams, /batch

The recommendation: **sequential headless pipeline** — `claude -p` with fresh context per feature, verification gates between each step

This is the "building the system that builds the system" moment — one more markdown file before any code runs

© Simple Technology Solutions, Inc.

The Actual Prompt

`

create a standalone research prompt for me and put the prompt in research/claude-code-agentic-rapid-development-best-practices-research-prompt.md

I will copy and paste that prompt into a claude.ai/chat using Opus 4.6 Extended with Research and Web Search checked then store the result .md file in this project.

given our tech stack and the functionality that we want to create research the current best ways to use claude code to rapidly create this functionality using spec driven development.

I'd prefer it if I could give the research/sts-demo-spec.md to claude code and it would just build it in one shot while I walk my dog (Milo - a cute 28lb Cavapoo). But I realize that claude code out of the box will not do that successfully, I first need to build an agentic harness/sdlc pipeline/agent teams/some-other-new-fangled-thing so that it can iteratively build out features of a certain size scope before continuing on to the next feature.

in the prompt give context of our situation and provide a recommendation

for the best way to rapidly build out this demo application.
`

- This prompt asked Claude Code to write a research prompt — a meta-prompt
- That research prompt was then pasted into claude.ai with web search for the actual research
- Result saved to: research/building-trailhead-autonomously-with-claude-code.md

The Research Prompt Claude Code Wrote

The generated research prompt included:
- Full situation context (7 features, 6 APIs, 8 hours remaining, tech stack)
- All Claude Code capabilities that could be part of the solution (subagents, agent teams, skills, hooks, headless mode, Agent SDK, /batch, CLAUDE.md, plan mode)
- Request to evaluate specific approaches with pros/cons
- Constraint: clean sequential git history (tells a better demo story than parallel branch merges)
- Request for an opinionated recommendation, not a menu of options

The Recommendation: Sequential Headless Pipeline

Claude's research evaluated 6 approaches and recommended:

Approach	Verdict	Why
Single long session	Rejected	Context degrades by feature 4, quality drops
Agent Teams	Rejected	Experimental, unstable, coordination overhead
/batch	Rejected	Can't handle shared dependencies between features
Agent SDK pipeline	Rejected	Overkill for this scope, more setup time than value
Custom subagents	Rejected	Unnecessary complexity for sequential work
Sequential headless	Chosen	Fresh context per feature, simple, reliable, clean git history

The Recommended Architecture

`
```
For each feature:
  1. claude -p "$(cat build/prompts/XX-feature.md)"    ← fresh context
  2. npx tsc --noEmit && pnpm build                    ← verification gate
  3. git add -A && git commit -m "feat: feature-name"   ← auto-commit on success
```
`

Key design decisions:
- Fresh context per feature — avoids degradation. Each call starts clean with only CLAUDE.md
- Verification gates — TypeScript compiler + build between every step. Deterministic check on probabilistic output
- Auto-commit on success — clean git history, one commit per feature
- CLAUDE.md under 80 lines — persistent memory across fresh contexts

- 11 self-contained prompts — each prompt file is a complete feature spec with "patterns to follow" pointers

Why This Step Is "Building the System That Builds the System"

- No code has been written yet
- We've produced 5 markdown files: STS briefing, API list, app spec, tech stack, build strategy
- Each one refined the last, adding specificity and making decisions
- The next step uses this research to create the actual build pipeline and 11 feature prompts
- Then the pipeline runs and produces the app — markdown in, software out

Step 7 — "Let's Start Implementing"

[simple]

One prompt to Claude Code: **"research complete, let's start implementing"** — it built the entire pipeline and ran it

Created `build/build.sh` (orchestration script), 11 feature prompts (`build/prompts/00–10`), and TRAILHEAD's CLAUDE.md

11 build steps, 0 failures — every step passed TypeScript compilation and build on the first try

Each step was a fresh `claude -p` call with its own whiteboard — feature 11 got the same quality as feature 1

Result: 4,455 lines of TypeScript, 58 files, 7 features, 20 git commits

© Simple Technology Solutions, Inc.

The Actual Prompt

```
research complete: research/building-trailhead-autonomously-with-claude-code.md
let's start implementing
```

That's it. Claude Code read the research result, then:
1. Created build/build.sh — the orchestration script with verification gates and auto-commit
2. Created 11 feature prompt files in build/prompts/
3. Created trailhead/CLAUDE.md — focused conventions under 80 lines
4. Ran all 11 build steps sequentially

The Build Script

```bash
build_step() {
  local step_name="$1"
  local prompt="$2"

  # Fresh context window for each feature
  claude -p "$prompt" \
```

```
  --allowedTools "Read,Write,Edit,Bash,Glob,Grep"

  # Verification gate
  npx tsc --noEmit
  pnpm build

  # Only commit on success
  git add -A && git commit -m "feat: $step_name"
}

build_step "scaffold"     "$(cat build/prompts/00-scaffold.md)"
build_step "shared-infra"  "$(cat build/prompts/01-shared-infra.md)"
# ... through 10-integration.md
`
```

What Each Build Step Produced

Step	Feature	What Was Built
00	Scaffold	Next.js 15 + Tailwind v4 + shadcn/ui (14 components), all dependencies via pnpm
01	Shared Infra	TypeScript types for 6 APIs, API client wrappers, 3 Zustand stores, TanStack Query provider, 25 curated federal land locations
02	Base Layout	Sidebar with nav links, searchable location selector grouped by agency, dashboard shell with placeholder cards
03	Weather	Open-Meteo API route + TanStack Query hook (15min auto-refresh) + dashboard card with weather icons
04	Seismic	USGS Earthquake API route + hook (5min refresh) + card with magnitude-coded badges
05	Fires	NASA FIRMS API route (CSV parsing) + hook (30min refresh) + card with alert banners + Haversine distance calc
06	Crime	FBI Crime Data API route + hook (24hr cache) + card with crime rate bars
07	Astronomy	Open-Meteo Astronomy API route + hook (1hr cache) + card with sunrise/sunset/moon phase/golden hour
08	Map	React Leaflet map (dynamic import, ssr:false), fire markers (red), earthquake markers (yellow), station pin, FlyTo animation, legend
09	Briefing	Claude API streaming briefing with 5-section system prompt, briefing history with expand/collapse, clipboard copy + print view
10	Integration	Watch list with live status cards, responsive sidebar→hamburger, error boundaries, welcome state

What a Build Prompt Looks Like

Each of the 11 prompts follows the same structure (~200-400 words):

```markdown
# Feature: USGS Earthquake Monitoring

## What to build
- API route at app/api/seismic/route.ts
- Earthquake list component with magnitude-coded badges
- TanStack Query hook in hooks/useSeismic.ts

## API Details
- Endpoint: https://earthquake.usgs.gov/fdsnws/event/1/query
```

- Parameters: format=geojson, latitude, longitude, maxradiuskm
- No authentication required

Patterns to follow ← THE KEY SECTION
- See lib/api/weather.ts for API client wrapper pattern
- See app/api/weather/route.ts for route handler pattern
- See hooks/useWeather.ts for TanStack Query hook pattern

Scope Control
- Implement ONLY what is specified above
- Do not add features beyond these requirements
`

Why It Worked

- "Patterns to follow" — points Claude to existing code. It reads those files first, then follows the same structure. This is how consistency survived across 11 fresh context windows.
- "API Details" — exact endpoints, parameters, auth requirements. No guessing, no hallucination.
- "Scope Control" — prevents Claude from adding auth, dark mode, i18n, extra error handling.
- Fresh context per step — no degradation. Step 10 gets the same quality of attention as step 1.
- Verification gates — tsc --noEmit && pnpm build between every step catches errors before they compound.

The Numbers

Metric	Value
Lines of TypeScript	4,455
Source files	58
Git commits	20
Build steps	11
Build failures	0
Human lines of code written	0

Step 8 — The Pipeline Built It. Then Reality Hit.

[simple]

The app compiled and built with zero failures — but **3 out of 6 APIs had real-world issues** when I opened the browser

Debug method: open app → see error → **save console log to a file → tell Claude "read this log"** → Claude diagnoses and fixes

NASA FIRMS: wrong URL format (lat/lng vs bounding box) — one prompt fix

FBI Crime Data: wrong env var name AND wrong endpoint format — two prompts, complete rewrite

Weather: defaulted to Celsius — one-line fix

AI Briefing: API key had zero credits — **not a code bug at all**

© Simple Technology Solutions, Inc.

The Debug Method

The debug loop was simple and repeatable:

1. Open the app in the browser
2. See something broken (empty data, error message, wrong values)
3. Open browser DevTools, copy the console log
4. Save it to trailhead/debug-logs/ (a gitignored directory Claude created for this purpose)
5. Tell Claude Code: "read this log"
6. Claude reads the log, diagnoses the issue, and fixes the code

This is the human-in-the-loop phase — the agent built the app, but real-world API integration still needed a human who could open a browser, see what's wrong, and point the agent at the right problem.

The Actual Prompts

Prompt 1 (Fire + Crime + Weather):
`
fire and crime data doesn't seem to be loading create a .gitignored folder for me to put the console log in and I will store it there
`

Then after saving the first log:
`
fire started to work still have issue with crime and also have an
error for weather. added console log
`

Prompt 2 (Briefing):
`
when I clicked generate briefing on the briefing page I got an error,
failed to fetch, here is the console log:
trailhead/debug-logs/localhost-1775157251165.log
`

Bug-by-Bug Breakdown

NASA FIRMS (Fire Data) — Wrong URL format
- Bug: Code used lat,lng,radius format in the API URL
- Reality: FIRMS expects bounding box format west,south,east,north
- Fix: One prompt. Claude rewrote the URL construction.
- Root cause: The model's training data had an outdated or incorrect FIRMS API pattern. Hallucination — it generated a plausible URL that wasn't correct.

FBI Crime Data — Two separate bugs
- Bug 1: Code read process.env.FBI_API_KEY but .env.local had FBI_CRIME_API_KEY
- Bug 2: Used endpoint /estimate/state/{state}?from=YYYY&to=YYYY which returns 404. The actual endpoint is /summarized/state/{state}/{offense-type}?from=MM-YYYY&to=MM-YYYY
- Fix: Two prompts. Complete rewrite of the crime API client.
- Root cause: Env var mismatch (simple typo) + hallucinated API endpoint format (the model generated a plausible endpoint that doesn't exist).

Weather — Wrong units
- Bug: Open-Meteo defaults to Celsius and km/h
- Fix: Added temperature_unit=fahrenheit and wind_speed_unit=mph to API params. One line.
- Root cause: Not a hallucination — the model just didn't think to specify units for a US-audience app.

AI Briefing — Not a code bug
- Bug: Browser showed ERR_EMPTY_RESPONSE. Server logs revealed: "Your credit balance is too low to access the Anthropic API."
- Fix: No code change. Bought API credits at console.anthropic.com.
- Root cause: Human forgot to fund the API key.

Why This Matters

- The pipeline worked perfectly — 11 build steps, 0 compilation failures, all code TypeScript-clean
- Real-world integration is a different problem — APIs have quirks, env vars can mismatch, third-party formats aren't always what the model expects
- The debug loop is fast — save a log, point Claude at it, get a fix. Each bug took 2-5 minutes to resolve.
- This is the "AI gets you 0-80%" lesson — the model built 4,455 lines of working TypeScript. The human identified 4 integration issues that required domain knowledge and a browser.

- Verification gates catch compile errors, not runtime errors — tsc --noEmit proves the types are correct, not that the API URL is right. Real-world testing is still the engineer's job.

Step 9 — Ship It

[simple]

Everything working locally. Asked Claude: **"what do I need to do to deploy to the internet?"**

Claude walked me through Vercel: install CLI, login, deploy — **three commands**

Added environment variables for the 3 APIs that need keys (NASA FIRMS, FBI Crime, Anthropic)

Production deploy: `vercel --prod --yes` — auto-detected Next.js, set root directory, built and deployed

Live URL: trailhead-pearl.vercel.app — from idea to deployed app in one afternoon

© Simple Technology Solutions, Inc.

The Actual Prompts

Prompt 1:
`
put credits on the anthropic api key everything is working locally.
what do I need to do to deploy to the internet?
`

Prompt 2 (after setting up Vercel account):
`
OK, I setup an account with vercel let install the cli
`

Prompt 3 (after logging in):
`
OK vercel logged in
`

The Deploy Sequence

1. Installed Vercel CLI locally (npm install --prefix ~/.local vercel)
2. Rudy logged in via vercel login (interactive — this was a human step)
3. First deploy with vercel --yes — auto-detected Next.js, set root to trailhead/
4. Added 3 environment variables via Vercel dashboard:

- NASA_FIRMS_MAP_KEY
- FBI_CRIME_API_KEY
- ANTHROPIC_API_KEY

5. Production deploy with vercel --prod --yes

What the Deploy Reveals

- Claude Code handled the deployment commands — it knows Vercel's CLI, knows how to set the root directory for a monorepo, knows the flags
- Interactive steps are still human — vercel login requires a browser OAuth flow. Claude can tell you to do it, but can't do it for you
- Environment variables are a human responsibility — API keys and secrets shouldn't be in the repo or in prompts. The engineer manages credentials.
- Zero-config for Next.js on Vercel — the tech stack decision (Step 4) paid off here. No Dockerfile, no nginx config, no build scripts. vercel --prod just works.

The Full Timeline

Time	Step	Tool
10:30 AM	Research: STS briefing	claude.ai
10:45 AM	Research: 100 free APIs	claude.ai
11:30 AM	Research: App spec (TRAILHEAD)	claude.ai
12:00 PM	Research: Tech stack	claude.ai
12:46 PM	Repo setup: ai-brown-bag.md	Claude Code
1:15 PM	Research: Build strategy	claude.ai
1:45 PM	Build: 11 features, 0 failures	Claude Code (headless)
3:05 PM	Run app, start debugging	Claude Code
3:10 PM	Fix fire + crime + weather APIs	Claude Code
3:20 PM	Fix briefing (buy API credits)	Human
3:30 PM	Deploy to Vercel	Claude Code + Human

Total elapsed time: ~5 hours
Human lines of code written: 0
Markdown files in the refinement chain: 6

Final Stats

Metric	Value
Lines of TypeScript	4,455
Source files	58
Git commits	20
Build steps	11
Build failures	0
API debug fixes	4
Deploy commands	3
Live URL	trailhead-pearl.vercel.app

[simple]

ACT 3

How to Improve Your Agentic Techniques

Now go do it — here's how to get better starting Monday.

© Simple Technology Solutions, Inc.

Stop Coding

[simple]

Your hands are no longer the best tool for writing code. **Models running on supercomputers are faster, more consistent, and don't get tired**

Engineering was never about typing — it was about solving problems. Now you solve problems by **directing agents instead of typing syntax**

Your new workflow: **prompt, review, refine** — not open file, type code, debug, repeat

This doesn't mean you stop thinking. It means you **stop being the bottleneck** between your ideas and working software

Go all in. Half-measures don't work — if you're still hand-writing boilerplate, you're leaving 10x on the table

© Simple Technology Solutions, Inc.

The Mindset Shift

This is the hardest tactic because it's not a technique — it's a habit change. Engineers have spent years building muscle memory for writing code. The instinct when you see a bug is to open the file and start typing. That instinct is now a bottleneck.

The old workflow:
1. Read the code
2. Understand the problem
3. Think about the fix
4. Type the fix
5. Run the build
6. See the error
7. Go back to step 3

The new workflow:
1. Tell the agent what's wrong
2. The agent reads the code, understands the problem, thinks about the fix, types the fix, runs the build, sees the error, and fixes it again
3. You review the result

Steps 1-6 of the old workflow are now steps the agent does in its loop. You do steps 1 and 3 of the new workflow — direction and review.

What "Stop Coding" Actually Means

It does NOT mean:
- Stop understanding code
- Stop reading code
- Stop making architectural decisions
- Stop reviewing what the agent produces
- Stop thinking about engineering problems

It DOES mean:
- Resist the urge to type code manually when an agent can do it
- Stop writing boilerplate by hand (CRUD routes, type definitions, test scaffolding)
- Stop context-switching between "thinking about the problem" and "typing the solution"
- Let the agent handle the mechanical translation from intent to implementation

Daily Actions

- Resist the urge to type code manually — describe what you want instead
- Go all in on agentic coding tools — Claude Code, claude.ai, the full pipeline
- Don't code, write prompts and review output
- Communicate to your agents what you want built — be an architect, not a typist

Why Half-Measures Don't Work

Engineers who use AI tools "sometimes" get marginal benefit. The compounding effect comes from:
- Building up CLAUDE.md with real conventions (only happens if you use it daily)
- Learning what makes a good prompt (only happens through repetition)
- Training your instinct to direct instead of type (only happens if you commit)
- Building a library of reusable prompts and templates (only happens over time)

The engineer who goes all in for a month learns 10x more than the one who tries it occasionally for six months.

Adopt Your Agent's Perspective

[simple]

Your agent is brilliant but **born blind** — every session starts with zero memory, zero context, zero understanding of your project

Before launching an agent, ask yourself: **"With what I'm giving it, could I do this job?"** If the answer is no, fix the inputs before you blame the output

The **Core Four** — every agent needs these to succeed: **Context** (what it knows), **Model** (how smart it is), **Prompt** (what you're asking), **Tools** (what it can do)

CLAUDE.md is how you give your agent your perspective — your conventions, your patterns, your "here's how we do things here"

Bad output is usually a **context problem**, not a model problem — give better inputs before reaching for a bigger model

© Simple Technology Solutions, Inc.

The Core Four: What Every Agent Needs

Every agent invocation has four levers you can adjust. When output quality is poor, diagnose which lever is the problem before retrying:

1. Context — Information & Perspective
- What the agent can "see": system prompt, CLAUDE.md, files it reads, conversation history, tool results
- This is the most common failure point. The agent doesn't know your project's conventions, your API's quirks, or your team's preferences unless you tell it
- Fix: Better CLAUDE.md, more targeted file reads in the prompt ("see lib/api/weather.ts for the pattern"), explicit constraints

2. Model — Intelligence & Potential
- Which model is running: Haiku (fast/cheap), Sonnet (balanced), Opus (deep reasoning)
- If the task is complex and Sonnet is struggling, upgrade to Opus. If it's simple and Opus is slow, drop to Haiku
- This is the lever people reach for first, but it's usually not the problem. A well-prompted Sonnet beats a poorly-prompted Opus

3. Prompt — Communication Method
- What you're asking the agent to do: the task description, constraints, scope, expected output

- Vague prompts produce vague results. "Fix the bug" vs "The weather API returns Celsius but should return Fahrenheit — update the Open-Meteo request params in lib/api/weather.ts to include temperature_unit=fahrenheit"
- Fix: Be specific about what to change, where to change it, and what patterns to follow

4. Tools — Agent Capabilities
- What the agent can interact with: Read, Write, Edit, Bash, Grep, Glob, MCP servers
- In headless mode, --allowedTools controls exactly which tools are available
- If the agent needs to run tests but doesn't have Bash access, it can't verify its own work
- Fix: Ensure the agent has the tools it needs for the full task cycle (implement + verify)

The Perspective Exercise

Before every prompt, mentally simulate:

> "I'm a senior engineer who just started at this company today. I've never seen this codebase. Someone hands me CLAUDE.md and this prompt. Can I complete this task?"

If the answer is no, you're missing context. Common things you know but the agent doesn't:
- Which file has the pattern to follow
- What the API endpoint actually returns
- Why a certain approach was chosen over alternatives
- What "done" looks like for this task

The 12 Leverage Points

Beyond the Core Four, there are 8 "through-agent" leverage points — things in your codebase that amplify the agent's effectiveness:

#	Leverage Point	What It Does
5	Standard Out	Command output the agent can read — build errors, test results, logs
6	Types	TypeScript types, schemas, error types — information flow the agent can follow
7	Documentation	Internal and 3rd-party docs the agent can reference
8	Tests	Self-validating feedback loops — the agent runs tests to check its own work
9	Architecture	Consistent codebase structure — patterns the agent can follow
10	Plans	Detailed prompts for massive work — break down complex tasks
11	Templates	Reusable embeddable prompts — solve problem classes, not individual problems
12	AI Developer Workflows	End-to-end automated pipelines — prompt to production

Each leverage point is something in your control that makes the agent more effective. The more of these you invest in, the more autonomous your agents become.

The Diagnosis Flowchart

When agent output is poor:

```
Bad output?
  ├──→ Was the prompt specific enough?
  |     No → Fix the prompt (Lever 3)
  |     Yes ↓
  ├──→ Did the agent have the right context?
  |     No → Add to CLAUDE.md or prompt (Lever 1)
  |     Yes ↓
  ├──→ Did the agent have the right tools?
  |     No → Add tools / permissions (Lever 4)
  |     Yes ↓
  └──→ Is the task too complex for this model?
       Yes → Upgrade model (Lever 2)
       No → Break the task into smaller pieces
```

Document Your App — Your Agent Reads It Too

[simple]

Your agent reads your docs the same way a new hire would. **If a new developer couldn't understand your app from the docs, neither can the agent**

Use **progressive disclosure** — CLAUDE.md has the 50 most important lines. Detailed docs live in markdown files the agent reads when it needs them

Don't cram everything into CLAUDE.md. **Point to deeper docs**: "For API patterns see docs/api-conventions.md" — the agent follows the link when relevant

READMEs, architecture docs, decision records, onboarding guides — **all of these are agent context now**, not just human reference

The documentation you wish you had when you joined the project? **Write it. Your agent needs it more than you did**

© Simple Technology Solutions, Inc.

Documentation as a Leverage Point

Documentation is one of the 12 leverage points of agentic coding. It serves a dual purpose:
- For humans: Onboarding, reference, decision history
- For agents: Context that supplements training data and CLAUDE.md

The agent's training data covers general knowledge (how React works, how REST APIs work). Your documentation covers project-specific knowledge (why you chose Zustand over Redux, how your API error handling works, what the data flow pattern is).

Progressive Disclosure for Agent Context

Not all documentation belongs in CLAUDE.md. The whiteboard is finite — every line in CLAUDE.md competes for attention on every single invocation. Instead, layer your documentation:

Layer 1: CLAUDE.md (~50-80 lines)
- Tech stack and versions
- Critical conventions (the ones that cause bugs if violated)
- Build/verification commands
- The 3-5 rules that apply to every task

Layer 2: Pointed references in CLAUDE.md
`markdown
Detailed Documentation
- API patterns: see docs/api-conventions.md
- Component patterns: see docs/component-guide.md
- Data flow: see docs/data-flow.md
- Testing: see docs/testing-strategy.md
`

The agent reads CLAUDE.md on every invocation but only reads the detailed docs when working on a relevant task. This keeps the whiteboard clean while making deep context available.

Layer 3: Inline documentation
- TypeScript types and interfaces (self-documenting)
- Code comments on non-obvious decisions
- README files in subdirectories

The agent discovers these naturally when it reads files in the relevant area of the codebase.

What to Document for Agents

Document	What It Contains	When the Agent Reads It
CLAUDE.md	Critical conventions, build commands, tech stack	Every session (automatic)
docs/api-conventions.md	API client pattern, route handler pattern, error handling	When building API features
docs/component-guide.md	Component structure, props patterns, state management	When building UI
docs/data-flow.md	How data moves: lib/api → route → hook → component	When debugging data issues
docs/decisions/	Architecture Decision Records (ADRs) — what was chosen and why	When making architectural changes
README.md per directory	What this directory contains and how it's organized	When exploring unfamiliar areas

The "New Hire" Test

Before starting a complex task, ask:

> "If I hired a senior engineer today and pointed them at this repo, could they complete this task from the docs alone?"

If the answer is no, the missing information is what you need to document. Common gaps:
- Why a pattern was chosen (not just what it is)
- Where the boundaries are between modules
- What the non-obvious conventions are (the ones that cause bugs when violated)
- How data flows through the system end-to-end

The Payoff

Documentation compounds:

- Write docs/api-conventions.md once → every API feature prompt can say "follow the pattern in docs/api-conventions.md" → consistent output across all features
- Write an ADR for a tech stack decision once → the agent never second-guesses the choice or introduces alternatives
- Write a component guide once → new components match existing patterns without per-prompt instructions

The upfront cost is one prompt to generate the doc (have Claude write it, you review). The ongoing benefit is better agent output on every future task in that area.

Daily Actions

- Keep CLAUDE.md tight — 50-80 lines of critical conventions only
- Create detailed docs in a docs/ directory and point to them from CLAUDE.md
- Use progressive disclosure: CLAUDE.md → pointed references → inline docs
- Apply the "new hire" test before complex tasks — if the docs aren't enough, fix the docs
- Have the agent write the documentation — it can read the codebase and produce docs faster than you

Don't Just Build — Iterate on What You Plan to Build

[simple]

The biggest mistake: jumping straight from idea to code. **Spec → Plan → Build.** Each step is a checkpoint where you catch bad ideas before they become bad code

A spec says **what** to build. A plan says **how** to build it. Both are markdown files the agent writes and you review — **iterate on the plan, not the implementation**

It's 100x cheaper to change a line in a spec than to rewrite a feature. **Catch the wrong API, the missing edge case, the scope creep in markdown — not in TypeScript**

Use Claude's plan mode: ask it to **plan first, then confirm before building.** Review the plan, ask questions, refine — then say "go"

The refinement chain is this principle applied to the whole project: **spec the spec, plan the plan, then build from a solid foundation**

© Simple Technology Solutions, Inc.

Why Spec-Driven Development Matters More with Agents

In traditional development, you can course-correct as you code — you notice a problem mid-implementation and adjust. With agents, especially in headless/out-of-the-loop mode, course correction is expensive:

- The agent builds exactly what you specified
- If the spec is wrong, you get a complete, well-built implementation of the wrong thing
- Fixing it means a new prompt, a new agent session, and potentially undoing work
- Each iteration costs tokens, time, and context

The fix: iterate on the spec, not the code. Changing a bullet point in a markdown spec costs nothing. Rewriting a feature costs everything.

The Three Stages

Stage 1: Spec — What to build

A spec answers:
- What features does this need?
- What data does it consume and produce?
- What does "done" look like?

- What is explicitly out of scope?

`markdown
Feature: Earthquake Monitoring

What it does
- Shows recent earthquakes near the selected location
- Color-coded by magnitude
- Auto-refreshes every 5 minutes

Data source
- USGS Earthquake API (free, no auth)
- Endpoint: earthquake.usgs.gov/fdsnws/event/1/query

Out of scope
- Historical earthquake data
- Push notifications
- Custom alert thresholds
`

Stage 2: Plan — How to build it

A plan answers:
- What files need to be created or modified?
- What existing patterns should be followed?
- What order should things be built in?
- What verification steps confirm it's done?

`markdown
Plan: Earthquake Monitoring

Files to create
1. lib/api/seismic.ts — API client wrapper (follow weather.ts pattern)
2. app/api/seismic/route.ts — Next.js route handler
3. hooks/useSeismic.ts — TanStack Query hook (5min refresh)
4. components/SeismicCard.tsx — Dashboard card

Build order
1. API client first (no dependencies)
2. Route handler (depends on API client)
3. Hook (depends on route)
4. Component (depends on hook)

Verification
- tsc --noEmit passes
- npm run build passes
- curl localhost:3000/api/seismic returns valid GeoJSON
`

Stage 3: Build — Execute the plan

Now the agent has a clear, reviewed spec and a concrete plan. The build prompt is essentially: "Execute this plan, following the patterns specified, and verify with these commands."

The Review Checkpoints

Each stage is a checkpoint where you catch problems cheaply:

Stage	What You Review	Cost of Change	Example Catch
Spec	Is this the right feature?	Free (edit markdown)	"We don't need push notifications for a demo"
Plan	Is this the right approach?	Free (edit markdown)	"Use the weather.ts pattern, not the crime.ts pattern — it's cleaner"
Build	Is this the right implementation?	Expensive (re-prompt, re-build)	"The API URL format is wrong" — now you're debugging

Problems caught at the spec stage cost nothing to fix. Problems caught at the build stage cost a full agent session. The earlier you catch them, the less you waste.

Using Claude's Plan Mode

Claude Code has a built-in plan mode for this workflow:

- /plan — Ask Claude to create a plan before acting. It will outline what it intends to do and wait for your approval.
- Review the plan — Ask questions, suggest changes, point out issues. The plan is just markdown in the conversation.
- Approve — Say "go" or "looks good" and Claude executes the plan.
- Multi-step planning — For complex tasks, have Claude plan the overall approach, then plan each step individually.

For headless pipelines, the plan lives in a markdown file:
- claude -p "Create a plan for this feature and save it to plans/seismic-plan.md"
- Review the plan file
- claude -p "Execute the plan in plans/seismic-plan.md"

The Iteration Principle

The refinement chain from Act 2 is this principle applied at project scale:

```
Idea → Spec (review) → Plan (review) → Build (verify)
```

At each arrow, you can loop back:
- Spec doesn't look right? Refine the spec before planning.
- Plan looks wrong? Revise the plan before building.
- Build fails? Check if the issue is in the plan or the spec first.

Never iterate on the implementation when you should be iterating on the plan. If the agent built the wrong thing, the fix is usually in the spec, not in the code.

Daily Actions

- Write a spec before every non-trivial task — even a 5-line bullet list
- Have the agent generate a plan, review it, then approve the build
- Keep specs and plans in the repo — they're documentation for what was

built and why
- When a build goes wrong, ask "was the spec right?" before asking "was the code right?"
- Use progressive refinement: rough spec → detailed spec → plan → build. Each step adds specificity

Structure Your Code, Enforce Standards

[simple]

Agents follow patterns. **The more consistent your codebase, the better the agent performs** — it reads existing code and mimics what it sees

Types are the agent's guardrails — **TypeScript strict mode catches the agent's mistakes at compile time** before they reach you

A consistent architecture means the agent doesn't have to guess. "See weather.ts for the pattern" works because every API follows the same structure

Linters, formatters, and type checkers are force multipliers — they're free feedback loops that run in milliseconds and catch entire categories of errors

Think of your codebase as the agent's training data for your project. **Messy code produces messy output**

© Simple Technology Solutions, Inc.

Why Structure Matters More for Agents Than for Humans

Human developers can navigate inconsistent codebases through tribal knowledge, Slack messages, and gut feeling. Agents have none of that — they have exactly what's on the whiteboard and what's in the files.

When your codebase has consistent patterns:
- The agent reads one example and applies it to new features correctly
- "See lib/api/weather.ts for the pattern" is a complete instruction
- New code looks like existing code — reviewable at a glance

When your codebase is inconsistent:
- The agent picks up conflicting patterns and produces a hybrid that matches nothing
- You spend review time explaining "no, not like that file, like this other file"
- The same prompt produces different quality depending on which files the agent happens to read first

Types as Agent Guardrails

TypeScript strict mode is the single highest-ROI investment for agentic coding:

- Compile-time error catching — tsc --noEmit catches type errors in under a second. The agent generates code, runs the compiler, reads the error, fixes it —

all in the agent loop.
- Self-documenting interfaces — When the agent reads a TypeScript interface, it knows exactly what shape data should take. No guessing, no hallucinating field names.
- Refactoring safety net — When the agent changes a type, the compiler immediately flags every file that needs updating. The agent can chase errors across the codebase automatically.
- Error types — Well-defined error types tell the agent what can go wrong and how to handle it.

```typescript
// This interface IS the documentation for the agent
interface WeatherResponse {
  temperature: number;      // Fahrenheit
  windSpeed: number;        // mph
  humidity: number;         // percentage
  conditions: WeatherCondition;
  forecast: DailyForecast[];
}
```

The agent reads this and knows exactly what to build. No ambiguity, no hallucination risk for the data shape.

Architecture as a Pattern Library

A consistent architecture means every new feature follows the same structure:

```
lib/api/weather.ts        → API client wrapper
app/api/weather/route.ts  → Next.js route handler
hooks/useWeather.ts       → TanStack Query hook
components/WeatherCard.tsx → UI component
```

When you add a new feature (seismic data), the agent reads the weather implementation and produces:

```
lib/api/seismic.ts
app/api/seismic/route.ts
hooks/useSeismic.ts
components/SeismicCard.tsx
```

The pattern carries itself. You don't need to specify file locations, naming conventions, or data flow — the agent infers it from the existing structure.

Enforcing Standards Automatically

Tool	What It Catches	Agent Integration
tsc --noEmit	Type errors, missing imports, wrong shapes	Agent runs it in the loop, reads errors, self-corrects
eslint	Style violations, unused vars, potential bugs	Agent can run and

auto-fix |
| prettier | Formatting inconsistencies | Agent can run as post-step |
| npm run build | Build errors, SSR issues, bundling problems | Verification gate between features |

These tools are deterministic checks on probabilistic output. They don't cost tokens, they run in milliseconds, and they catch entire categories of errors that the model might miss.

Daily Actions

- Use TypeScript strict mode — never any, never @ts-ignore unless absolutely necessary
- Maintain consistent file and directory structure across features
- Add type definitions for all API responses, component props, and store state
- Run linters and formatters automatically (pre-commit hooks or in the agent's verification step)
- When you see inconsistency, refactor it — you're improving the agent's pattern library

Your Codebase Is Growing — Research How to Handle It

[simple]

When your app gets more complex, **don't guess at best practices — research them.** Use the markdown refinement technique: write a research prompt with context about your codebase, run it in claude.ai with web search, save the result

The research prompt includes **what your codebase looks like today** — tech stack, file count, pain points, what's getting unwieldy. Claude searches for how other teams solved the same problem at this scale

The result is a **best practices document tailored to your situation** — not generic advice, but "given your Next.js app with 58 files and 6 API integrations, here's how to handle state management as you grow"

Use that document to **update your standards and refactor** — it becomes the new input to CLAUDE.md, architecture docs, and refactoring prompts

This is the refinement chain applied to maintenance: **codebase context → research prompt → best practices → updated standards → agent-driven refactor**

© Simple Technology Solutions, Inc.

The Problem: Complexity Creeps In

Every codebase hits inflection points where what worked at 10 files breaks at 100:

- State management that was fine with 3 stores gets tangled at 10
- API routes that were simple with 2 integrations get inconsistent at 8
- Component patterns that worked for a dashboard don't scale to multi-page flows
- Error handling that was ad-hoc needs standardization
- Testing that was optional becomes necessary

The instinct is to Google it, read a few blog posts, and wing it. The agentic approach: create a research prompt with your specific context and get a tailored recommendation.

The Technique: Research Prompt with Codebase Context

Step 1: Write a research prompt that describes your situation

Have Claude Code write this for you. It can read the actual codebase and generate an accurate description:

`

Create a standalone research prompt and save it to
research/state-management-best-practices-prompt.md

I will paste this into claude.ai with web search enabled.

Context: our app has grown from 7 features to 15. We have 3 Zustand stores that are starting to overlap. Some components reach into multiple stores. The data flow pattern (lib/api → route → hook → component) is getting complex with cross-feature dependencies.

Research: what are the current best practices for state management in a Next.js 15 App Router application at this scale? Should we restructure our stores? Add a state machine? Use a different pattern?

Be opinionated — I want "here's what you should do" not "5 options."
`

Step 2: Run the research in claude.ai

- Paste the prompt into claude.ai with Opus 4.6 Extended + Research + Web Search
- Claude searches the web for current best practices, reads documentation, and synthesizes a recommendation
- Save the result as a markdown file in your repo: research/state-management-best-practices.md

Step 3: Use the result to update standards

The research document becomes input for:
- CLAUDE.md updates — new conventions, updated patterns
- Architecture docs — updated decision records explaining why the approach changed
- Refactoring prompts — specific instructions for the agent to migrate existing code

Step 4: Agent-driven refactor

`
Refactor our Zustand stores following the recommendations in
research/state-management-best-practices.md

Specifically:
- Split the location store into location-selection and location-data
- Extract shared data fetching logic into a unified hook
- Update all components that import from the old stores

Verify: tsc --noEmit && npm run build
`

Examples of When to Use This

Inflection Point	Research Prompt Context
State management getting tangled	"We have 3 Zustand stores, components reaching across stores, cross-feature dependencies"
API error handling inconsistent	"6 API routes with 3 different error

handling patterns, no standard error types" |
Testing strategy needed	"58 files, 0 tests, about to add authentication — what's the highest-ROI testing strategy?"
Performance degradation	"Dashboard loads 5 API routes on mount, render time increasing, no caching strategy"
Authentication being added	"Currently no auth, need to add role-based access for federal users, Next.js App Router"
Monorepo structure needed	"Single app growing into shared components used by 2 apps, need to split without breaking"

Each of these is a standalone research prompt with specific codebase context that produces a tailored best practices document.

Why This Beats Generic Advice

Generic advice: "Use Zustand for client state in Next.js apps."

Researched-for-your-codebase advice: "Given your 3 overlapping Zustand stores and the pattern where WeatherCard and SeismicCard both read from locationStore, split locationStore into selection state (which location is active) and derived data state (what data has been fetched). Use TanStack Query's dependent queries to cascade data fetching from selection changes. Here's the migration path..."

The difference is specificity. The research prompt encodes your exact situation, and the result addresses your exact problems.

The Refinement Chain for Maintenance

```
Codebase context (what it looks like now)
  → Research prompt (.md) — Claude Code writes this from reading the code
    → Best practices result (.md) — claude.ai researches and recommends
      → Updated standards (.md) — CLAUDE.md, architecture docs, conventions
        → Refactoring prompts (.md) — specific migration instructions
          → Agent-driven refactor — Claude Code executes the migration
```

This is the same refinement chain from Act 2, applied to an existing codebase instead of a greenfield build. Each step produces a markdown artifact that feeds the next step.

Daily Actions

- When you hit a complexity wall, don't wing it — write a research prompt with your codebase context
- Have Claude Code write the research prompt — it can read the actual code and describe the situation accurately
- Run the research in claude.ai with web search for current best practices
- Save results in research/ and use them to update CLAUDE.md and architecture docs
- Use the updated standards as input for agent-driven refactoring

Capture Tech Debt, Refactor Often

[simple]

Every time the agent produces something that "works but isn't right," **capture it** — don't just move on. Tech debt compounds faster when agents generate at scale

Agents are **cheap refactoring machines** — what used to be a half-day chore is now a prompt. "Refactor all API routes to use the new error handling pattern" takes minutes

Keep a running list: `TODO.md`, GitHub issues, inline `// TODO:` comments — give the agent a backlog it can work through

Refactoring improves future agent output — every inconsistency you fix is one fewer bad pattern the agent might copy

Schedule refactoring like you schedule features. **An hour of agent-driven refactoring per week** keeps the codebase clean and the agent effective

© Simple Technology Solutions, Inc.

Why Tech Debt Matters More in Agentic Codebases

In traditional development, tech debt slows down humans — but humans can work around it with tribal knowledge. In agentic development, tech debt actively degrades agent output:

- Agents copy what they see. If three API routes use different error handling patterns, the agent picks one — maybe the worst one.
- Inconsistency confuses "patterns to follow" instructions. If you say "see weather.ts for the pattern" but weather.ts uses a deprecated approach, the agent propagates that debt.
- Agents generate at scale. A human might write one bad pattern. An agent told to build 7 features from that pattern produces 7 copies of the debt.

The cost of tech debt is no longer just "slower development" — it's "worse agent output across the entire codebase."

Agents as Refactoring Machines

The economics of refactoring have fundamentally changed:

Before agents:
- "Rename all instances of userId to accountId across 40 files" = half a day of tedious find-and-replace with manual verification

- "Migrate all API routes from Express to Next.js route handlers" = multi-day effort
- Result: refactoring gets deferred, debt accumulates

With agents:
- "Rename all instances of userId to accountId across the codebase, update all types, tests, and imports" = one prompt, 5 minutes
- "Refactor all API routes to use the new error handling pattern from lib/api/weather.ts" = one prompt, 15 minutes
- Result: refactoring is cheap enough to do continuously

How to Capture Debt for Agents

TODO.md in the project root:
`markdown
Tech Debt Backlog
- [ ] Migrate crime API route to new error handling pattern
- [ ] Extract location selector into shared component
- [ ] Add TypeScript strict null checks to legacy modules
- [ ] Standardize date formatting across all API responses
`

The agent can read this file and work through items. Each item is a self-contained prompt.

Inline // TODO: comments:
`typescript
// TODO: This uses the old error handling pattern — refactor to match weather.ts
`

The agent finds these with Grep and can fix them systematically.

GitHub Issues:
- Label issues with tech-debt or refactor
- Include enough context that the agent can pick up the issue and work on it
- This is the beginning of the PITER framework — issues as prompt input

The Refactoring Cadence

- After every feature: Quick pass — did the agent introduce any inconsistencies? Fix them now before the next feature copies them.
- Weekly: Dedicated refactoring session. Give the agent the TODO.md backlog and let it work through items.
- Before major features: Clean up the area of the codebase the new feature will touch. The agent's output quality depends on what it reads.

Daily Actions

- Maintain a running tech debt backlog (TODO.md, issues, or inline comments)
- After each agent session, review for patterns that "work but aren't right"
- Use the agent for refactoring — it's fast and cheap, there's no excuse to defer
- Treat refactoring as improving the agent's pattern library, not just cleaning up code
- Fix inconsistencies before they multiply — one bad pattern becomes seven if

you wait

Your Goal: Get Out of the Loop

[simple]

Three levels of agentic coding: **In-the-loop** (you prompt, review every step) → **Out-of-the-loop** (agent works autonomously, you review at the end) → **Zero-touch** (prompt to production, your review becomes the bottleneck)

Your KPI: **how many touchpoints does a task need?** In-the-loop = many. Out-of-the-loop = two (prompt + review). Zero-touch = one (prompt only)

Moving up the ladder requires **trust, and trust requires verification** — you can't skip the loop until verification gates prove the agent is reliable

Every tactic in this presentation is designed to **push you toward out-of-the-loop** — better CLAUDE.md, consistent architecture, verification gates, feedback loops

Start by identifying one repeating task you do every week and ask: **what would it take to make the agent do this without me watching?**

© Simple Technology Solutions, Inc.

The Three Levels of Agentic Coding

Level 1: In-the-Loop
- You prompt the agent, watch it work, approve or correct each step
- Every tool call gets your review. Every file write needs your OK.
- This is where everyone starts. It builds understanding and trust.
- Touchpoints per task: many (5-20+)
- Analogy: pair programming where you're the driver and the AI is the navigator — except you keep switching seats

Level 2: Out-of-the-Loop (AFK)
- You write a prompt, kick off the agent, and walk away
- The agent works autonomously: reads code, writes code, runs builds, fixes errors, commits
- You come back and review the result — not the process
- Touchpoints per task: two (prompt + review)
- Analogy: delegating to a competent junior — you give clear instructions, they execute, you review the PR
- This is the TRAILHEAD model: claude -p with a feature spec, verification gates, auto-commit

Level 3: Zero-Touch
- A trigger (GitHub issue, scheduled job, webhook) launches the agent
- The agent completes the task end-to-end and opens a PR or deploys

- Human review becomes optional — the verification gates are trusted enough
- Touchpoints per task: one (prompt only) or zero (trigger-based)
- Analogy: CI/CD for AI — just as you stopped manually deploying, you stop manually prompting
- This is the PITER framework: Prompt Input → Trigger → Environment → Review

The PITER Framework for Out-of-the-Loop Agents

Moving from in-the-loop to out-of-the-loop requires four things:

Pi — Prompt Input
- Where the work request comes from: GitHub issues, JIRA tickets, Slack messages, scheduled triggers
- The prompt must be complete enough for the agent to work without follow-up questions

T — Trigger
- What launches the agent: webhook on issue creation, cron schedule, manual dispatch
- Claude Code supports scheduled tasks, GitHub Actions integration, and API-triggered runs

E — Environment
- Where the agent runs: isolated git worktree, dedicated branch, sandboxed container
- Must be safe for unsupervised execution — can't accidentally push to main or delete production data

R — Review
- How you validate the output: PR review, test results, deployment preview
- This is the last human touchpoint — and the goal is to make it so reliable that even this becomes lightweight

Measuring Progress: Agentic Coding KPIs

KPI	In-the-Loop	Out-of-the-Loop	Zero-Touch
Touchpoints per task	5-20+	2	0-1
Human time per feature	Hours	Minutes (prompt + review)	Seconds (glance at PR)
Agent autonomy	Low	High	Full
Trust requirement	Low	Medium	High
Verification investment	Manual review	Automated gates	Automated gates + tests

How to Move Up the Ladder

From in-the-loop to out-of-the-loop:
1. Write a complete CLAUDE.md with conventions and verification commands
2. Create detailed feature specs with "patterns to follow" and scope control
3. Add verification gates (tsc --noEmit && npm run build) after every step
4. Try claude -p with a feature prompt and review the result instead of watching the process
5. If the result is good, you're out of the loop for that type of task

From out-of-the-loop to zero-touch:
1. Template your prompts — turn one-off specs into reusable meta-prompts
2. Set up triggers — GitHub issues, cron jobs, webhooks that launch agents
3. Invest in tests — the agent needs automated validation, not human eyes
4. Build confidence incrementally — start with low-risk tasks (docs, tests, refactors) before high-risk ones (features, deployments)

Daily Actions

- Identify one repeating task and try to move it one level up the ladder
- Track your touchpoints — how many times did you intervene on the last task?
- Invest in verification gates — they're the bridge between "I need to watch" and "I can walk away"
- Use the PITER framework to design your first out-of-the-loop workflow
- Accept that the goal isn't to eliminate humans — it's to eliminate busywork so you focus on judgment

Have Claude Run Your Tools, Not You

[simple]

Claude Code has a real terminal — it can run **any command you can**: builds, tests, linters, curl, git, database queries. Stop copying output and pasting it back in

When the agent runs `npm run build` and gets an error, **the error goes straight into the whiteboard** — the agent reads it and fixes it in the same loop. No human middleman

The verification gate pattern: `tsc --noEmit && npm run build` after every change. **The compiler is the agent's spell-checker** — deterministic, instant, and always right

Don't describe the error to Claude — make Claude see the error itself. "Run the tests" beats "here's what the test output said" every time

If you're running a command, reading the output, and then telling Claude what happened — **you're doing the agent's job for it**

© Simple Technology Solutions, Inc.

Standard Out as a Leverage Point

Standard out (stdout/stderr) is one of the 12 leverage points of agentic coding. It's the bridge between the agent's probabilistic reasoning and the deterministic reality of your tools.

When the agent runs a command via the Bash tool:
1. The command executes in a real shell
2. The output (stdout and stderr) returns to the agent's context
3. The agent reads the output and reasons about what happened
4. Based on the result, it decides what to do next

This creates a tight feedback loop that doesn't require human involvement. The agent writes code, runs the build, reads the errors, fixes them, and runs again — all within a single session.

Commands the Agent Should Be Running

Command	What It Tells the Agent
tsc --noEmit	Type errors with exact file, line, and expected type
npm run build	Build failures, missing imports, SSR issues
npm test	Test failures with assertion details
eslint .	Style violations and potential bugs

git status	What's changed, what's staged, what's untracked
git diff	Exact changes made — the agent can review its own work
curl localhost:3000/api/weather	API response shape — does the route work?
cat debug-logs/error.log	Runtime errors from the running application

The Anti-Pattern: Human as Clipboard

A common anti-pattern in early agentic adoption:

```
Human: runs npm run build
Human: sees error "Module not found: @/components/WeatherCard"
Human: types to Claude "I got an error about WeatherCard module not found"
Claude: "Try importing from the correct path..."
```

What should happen:

```
Human: "Run the build"
Claude: [runs npm run build]
Claude: [reads error: Module not found: @/components/WeatherCard]
Claude: [checks the actual file location with Glob]
Claude: [fixes the import path]
Claude: [runs build again — passes]
```

In the first version, the human is the clipboard between the terminal and the agent. In the second version, the agent handles the entire loop. The human initiated the task and reviewed the result — everything in between was the agent's job.

The Verification Gate Pattern

The most important use of standard out is the verification gate:

```bash
# After every code change:
npx tsc --noEmit    # Does it type-check?
npm run build       # Does it build?
npm test            # Do tests pass?
```

Why this matters:
- The model is probabilistic — it can generate code that "looks right" but has type errors
- The compiler is deterministic — it catches every type error in under a second
- When the agent runs both, it gets instant, reliable feedback on its own work
- This is the "deterministic check on probabilistic output" principle

The key insight from TAC: "AI agents are probabilistic systems making deterministic claims. An agent can conclude its code is correct — while tsc catches three type errors in under a second."

Making Logs Visible to the Agent

Sometimes the agent can't run the app itself (browser-based errors, UI issues). In these cases, bring the output to the agent:

- Console logs: Save browser console output to a file, tell the agent to read it
- Server logs: Point the agent at log files in the project
- Screenshots: Claude Code can read images — take a screenshot of the error and show it
- Debug directories: Create a gitignored debug-logs/ directory for dumping runtime output

The principle: whatever you can see, make the agent see it too. The less you're the middleman between the error and the fixer, the faster the loop runs.

Daily Actions

- Let the agent run builds, tests, and linters — don't run them yourself and report the results
- Add verification commands to CLAUDE.md so the agent knows to run them automatically
- Create a debug-logs directory for browser/runtime output the agent can't see directly
- When you catch yourself copying terminal output into a prompt, stop — let the agent run the command itself
- Think of every command-line tool as a potential feedback loop for the agent

Claude Makes Mistakes — Make It Find Them Before You Do

[simple]

Your work is useless unless it's tested. **Add validation to every agentic prompt** — don't just say "build the feature," say "build the feature and verify it compiles"

The magic of agents: they can **write code, test it, see the failure, and fix it** — all in one loop, without you lifting a finger

Stack your feedback loops: **type checker → build → lint → tests → runtime check**. Each layer catches what the previous one missed

The more feedback loops you add, the **less you need to review** — this is what moves you from in-the-loop to out-of-the-loop

Spending compute on testing increases your confidence. **A $0.50 test run is cheaper than 30 minutes of your review time**

© Simple Technology Solutions, Inc.

The Closed-Loop Prompt Pattern

The most important prompt design pattern: request → validate → resolve

Open-loop prompt (bad):
`
Build the seismic data feature with a USGS API route and dashboard card.
`

Closed-loop prompt (good):
`
Build the seismic data feature with a USGS API route and dashboard card.

After implementation:
1. Run npx tsc --noEmit — fix any type errors
2. Run npm run build — fix any build errors
3. Verify the API route returns valid JSON: curl localhost:3000/api/seismic
`

The difference: the open-loop prompt trusts the agent's self-assessment ("I think this is correct"). The closed-loop prompt forces the agent to prove it with deterministic tools.

Stacking Feedback Loops

Each feedback loop catches a different category of error:

Layer	Tool	What It Catches	Speed
1. Types	tsc --noEmit	Wrong types, missing imports, shape mismatches	< 1 second
2. Build	npm run build	SSR issues, bundling errors, missing modules	5-15 seconds
3. Lint	eslint .	Style violations, unused vars, potential bugs	2-5 seconds
4. Tests	npm test	Logic errors, regression bugs, wrong behavior	10-60 seconds
5. Runtime	curl / browser	API format issues, wrong data, integration bugs	Manual

Layers 1-4 can all be automated in the agent's loop. Layer 5 requires a human (or a browser automation tool like Playwright MCP).

The key insight: each layer is cheap. Running all four automated layers costs maybe 30 seconds and a few cents of compute. Skipping them and relying on human review costs 15-30 minutes of your time.

How Feedback Loops Enable Autonomy

The relationship between feedback loops and autonomy:

```
No feedback loops    → You must review every line          → In-the-loop
Type checker only    → You review for logic, not syntax    → Partially out
Types + build + lint → You review for design, not errors   → Mostly out
Types + build + tests → You review for intent, not correctness → Out-of-the-loop
Full CI/CD pipeline  → You review PRs, not code            → Approaching zero-touch
```

Every feedback loop you add is a category of errors you no longer need to check manually. This is the mechanism that moves you up the autonomy ladder.

The Economics of Agent Testing

```
Scenario A: No feedback loops
  - Agent generates feature
  - You review 200 lines of code manually: 20-30 minutes
  - You find 3 bugs, describe them to the agent
  - Agent fixes, you review again: 10 minutes
  - Total: 30-40 minutes of your time

Scenario B: Full feedback loops
  - Agent generates feature
  - Agent runs tsc: catches 2 type errors, fixes them
  - Agent runs build: catches 1 SSR issue, fixes it
  - Agent runs tests: all pass
```

```
  - You review the committed code: 5 minutes (confidence is high)
  - Total: 5 minutes of your time + $0.50 compute
```

The agent spent an extra 2-3 iterations and maybe $0.50 in tokens to run those feedback loops. You saved 25-35 minutes. That tradeoff gets better every single time.

Building Feedback Loops into CLAUDE.md

Add verification commands directly to CLAUDE.md so the agent runs them automatically:

```markdown
## Verification (run after every feature)
npx tsc --noEmit && npm run build

## Testing (run after logic changes)
npm test

## Linting (run before committing)
eslint . --fix
```

When verification is in CLAUDE.md, the agent treats it as a convention — it will run these commands as part of its workflow without being asked in every prompt.

Daily Actions

- Add validation commands to every agentic prompt — never just "build it," always "build it and verify"
- Create closed-loop prompts: request, validate, resolve
- Set up end-to-end tests for high-ROI agent self-validation
- Put verification commands in CLAUDE.md so they run automatically
- Chain feedback mechanisms: types → build → lint → tests
- Accept that spending compute on testing is an investment in your time

From Writing Code to Directing Agents Who Write Code

[simple]

The SDLC still exists — research, spec, build, test, deploy — but **who does what changes.** You specify intent. Agents generate implementation

Template your engineering — when you solve a problem with an agent, turn that prompt into a reusable template. Next time, it's a one-click solution

One agent, one prompt, one purpose. **Don't cram everything into one session** — specialized agents with focused prompts produce better results than one overloaded generalist

The progression: start **in-the-loop** (learn the tools) → move to **out-of-the-loop** (trust the verification) → target **zero-touch** (prompt to production)

Start Monday: pick one repeating task, write a CLAUDE.md, add a verification gate, and **let the agent do it while you do something else**

© Simple Technology Solutions, Inc.

The Agentic SDLC

Traditional SDLC phases still exist, but the human/agent roles shift:

Phase	Traditional	Agentic
Research	Human reads docs, browses web	Agent searches web, synthesizes findings into markdown
Spec	Human writes requirements	Human directs agent to write spec, reviews and refines
Architecture	Human designs system	Human defines constraints, agent proposes architecture
Build	Human writes code	Agent writes code, runs verification, auto-commits
Test	Human writes and runs tests	Agent writes tests, runs them, fixes failures
Debug	Human reads errors, fixes code	Agent reads errors, diagnoses, fixes — human handles edge cases
Deploy	Human runs deploy commands	Agent runs deploy commands, human manages credentials
Review	Human reviews PRs	Human reviews agent output — the last touch point

The engineer's role evolves from code producer to architect, reviewer, and

system designer. Your value shifts from typing to judgment.

Template Your Engineering

When you solve a problem with an agent, you've created a recipe. Template it:

Level 1: Save the prompt
`markdown
Bug Fix Template
Read the error log at {path}. Diagnose the root cause.
Fix the issue. Run tsc --noEmit && npm run build to verify.
`

Level 2: Create a parameterized template
`markdown
New API Feature Template
Build an API route at app/api/{feature}/route.ts
Create a TanStack Query hook at hooks/use{Feature}.ts
Follow the pattern in lib/api/weather.ts
Run verification: tsc --noEmit && npm run build
`

Level 3: Meta-prompts that generate prompts
`markdown
Given the feature spec below, generate a build prompt that:
- Lists specific files to create
- Points to existing patterns to follow
- Includes scope control
- Includes verification commands
`

Each level increases reusability. Level 3 is where you start building the system that builds the system.

One Agent, One Prompt, One Purpose

From the TAC framework — massive context windows lead to distracted, confused agents. The solution:

- Dedicated agents for each workflow step — research agent, build agent, test agent, deploy agent
- Focused prompts — each prompt has a single purpose, not "also fix the CSS and update the docs"
- Free up the whiteboard — a focused prompt leaves most of the context window available for the actual work
- Commit and version control all prompts — they're engineering artifacts, treat them like code

The headless pipeline pattern embodies this: each claude -p call is one agent, one prompt, one purpose. Feature 1 gets its own session, Feature 2 gets its own session. No cross-contamination.

The AI Developer Workflow (ADW)

The end-state is a fully automated pipeline — the AI Developer Workflow:

```
Trigger (GitHub issue, cron, webhook)
  → Agent reads the issue/task description
  → Agent researches the codebase
  → Agent implements the solution
  → Agent runs verification gates
  → Agent opens a PR / deploys to preview
  → Human reviews (optional)
```

This is the PITER framework in production:
- Prompt Input — Issues, tickets, or scheduled tasks
- Trigger — Webhook, cron job, or GitHub Action
- Environment — Isolated worktree or branch for safe execution
- Review — PR review, automated test results, deployment preview

Getting Started: Your First Week

Monday: Install Claude Code. Write your first CLAUDE.md with project conventions and a verification command.

Tuesday: Take a task you'd normally hand-code and try prompting Claude Code instead. Stay in-the-loop — watch what it does.

Wednesday: Try a headless prompt: claude -p "$(cat your-prompt.md)". Review the result instead of watching the process.

Thursday: Identify something you do every week (update dependencies, write a test, fix a lint error) and write a reusable template for it.

Friday: Run the template. Review the result. If it's good, you just automated one task. Repeat next week with another one.

Daily Actions

- Build higher-order prompts that pass plans into plans
- Create specialized templates for engineering tasks (bugs, features, refactors, chores)
- Review and refine templates based on execution outcomes
- Scale from single agent to multi-agent workflows as confidence grows
- Track your KPIs: touchpoints per task, time spent reviewing, autonomous success rate

Get Started

[simple]

The App: trailhead-pearl.vercel.app

The Repo: github.com/keepitsts/claude-ai-brown-bag — every prompt, every research artifact, every build script is in the repo

Want Claude Code? If you're doing dev work where AI is allowed, you can get a license. Request it on the service desk:

keepitsts.atlassian.net/servicedesk/customer/portal/1/group/103/create/180

© Simple Technology Solutions, Inc.

Links

- Live app: https://trailhead-pearl.vercel.app
- GitHub repo: https://github.com/keepitsts/claude-ai-brown-bag
- Claude Code license request: https://keepitsts.atlassian.net/servicedesk/customer/portal/1/group/103/create/180

Getting a Claude Code License

Aaron Kilinski's guidance: "Rule of thumb is that if the employee is going to use it for their work (would need to be able to describe that) then they can get a license. For instance, anyone doing dev work (where AI is allowed) would be approved."

Tim Harper has added a request form to the service desk. Use the link above to submit your request.

What's in the Repo

The repo is a complete record of everything in this presentation:

- prompts/ — Every prompt given to Claude, numbered sequentially (001–017)
- research/ — All research artifacts (STS briefing, API list, demo spec, tech

stack, build strategy)
- build/prompts/ — The 11 headless build prompts (00-scaffold through 10-integration)
- build/build.sh — The orchestration script with verification gates
- trailhead/ — The complete application source code (4,455 lines, 58 files)
- trailhead/CLAUDE.md — The conventions file that guided every build step
- presentation/ — The slides and research for this presentation

The entire process is transparent and reproducible.